notebook

April 28, 2026

1 Regression

1.1 Imports

[1]:
```
import time
import math
import copy

from IPython.display import display
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
%matplotlib widget
plt.rcParams['figure.figsize'] = (12, 8)
plt.rcParams["figure.constrained_layout.use"] = True

from sklearn.metrics import (
    mean_squared_error, mean_absolute_error,
    r2_score,
)
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.model_selection import GridSearchCV

from regression.base.basemodel import BaseModel
from regression.base.basegdmodel import BaseGDModel
from regression.dataset import CaliforniaHousingDataset as Dataset
from regression.pipeline import get_pipeline
from regression.eda.eda import EDA
from regression.logger import get_logger

# Linear regression
from regression.linear_regression.diagnostics import GaussMarkovDiagnostics
from regression.linear_regression.ols import OLS
from regression.linear_regression.wls import WLS
from regression.linear_regression.mbgd import MBGD

# Regularization and features selection
```

```
from regression.regularization.ridge_regression import RidgeRegression,␣
 ↪RidgeRegressionCV
from regression.regularization.lasso_regression import LassoRegression,␣
 ↪LassoRegressionCV
from regression.regularization.elastic_net import ElasticNet, ElasticNetCV
from regression.features_selection.features_selection import \
    forward_selection, backward_elimination, lasso_selection

# Nonlinear basis
from regression.nonlinear_basis.polynomial import PolynomialBasis
from regression.nonlinear_basis.fourier import FourierBasis
from regression.nonlinear_basis.rbf import RBF
from regression.nonlinear_basis.validation_curve import ValidationExperiment
from regression.nonlinear_basis.ablation_study import AblationStudy

# Advanced
from regression.advanced.gpr import GPR
from regression.advanced.irls import IRLS
from regression.advanced.kernel_ridge import KernelRidgeCV
from regression.advanced.bias_variance import BiasVarianceAnalyzer
from regression.advanced.bayes_reg import BayesianLinearRegression

# Evaluation
from regression.evaluation.visualizer import Visualizer
from regression.evaluation.evaluator import Evaluator

# Sensitivity & Robustness
from regression.sensitivity_robustness import (
    sensitivity_split_analysis,
    plot_split_boxplots,
    noise_injection_analysis,
    feature_corruption_analysis,
    plot_loss_curves
)

# Benchmark
from regression.benchmark import benchmark_models
```

1.2 Dataset

[2]:
```
d = Dataset()
d.split()
```

[3]:
```
eda = EDA(d)
```

1.2.1 Missing values

[4]:
```
display(eda.missing_values())
```

	feature	missing_count	missing_percent
MedInc	MedInc	0	0.0
HouseAge	HouseAge	0	0.0
AveRooms	AveRooms	0	0.0
AveBedrms	AveBedrms	0	0.0
Population	Population	0	0.0
AveOccup	AveOccup	0	0.0
Latitude	Latitude	0	0.0
Longitude	Longitude	0	0.0
MedHouseVal	MedHouseVal	0	0.0

1.2.2 Descriptive statistics

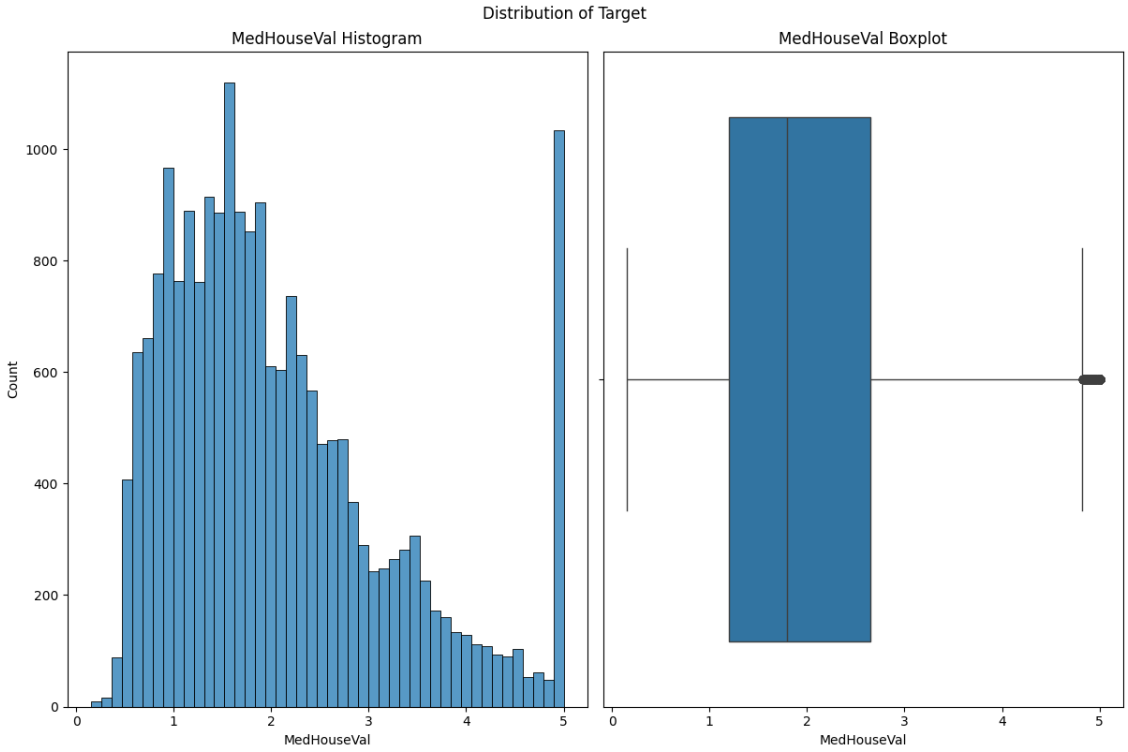
[5]:
```
display(eda.descriptive_stats())
```

	MedInc	HouseAge	AveRooms	AveBedrms	Population	\
count	20640.000000	20640.000000	20640.000000	20640.000000	20640.000000	
mean	3.870671	28.639486	5.429000	1.096675	1425.476744	
std	1.899822	12.585558	2.474173	0.473911	1132.462122	
min	0.499900	1.000000	0.846154	0.333333	3.000000	
25%	2.563400	18.000000	4.440716	1.006079	787.000000	
50%	3.534800	29.000000	5.229129	1.048780	1166.000000	
75%	4.743250	37.000000	6.052381	1.099526	1725.000000	
max	15.000100	52.000000	141.909091	34.066667	35682.000000	

	AveOccup	Latitude	Longitude	MedHouseVal
count	20640.000000	20640.000000	20640.000000	20640.000000
mean	3.070655	35.631861	-119.569704	2.068558
std	10.386050	2.135952	2.003532	1.153956
min	0.692308	32.540000	-124.350000	0.149990
25%	2.429741	33.930000	-121.800000	1.196000
50%	2.818116	34.260000	-118.490000	1.797000
75%	3.282261	37.710000	-118.010000	2.647250
max	1243.333333	41.950000	-114.310000	5.000010

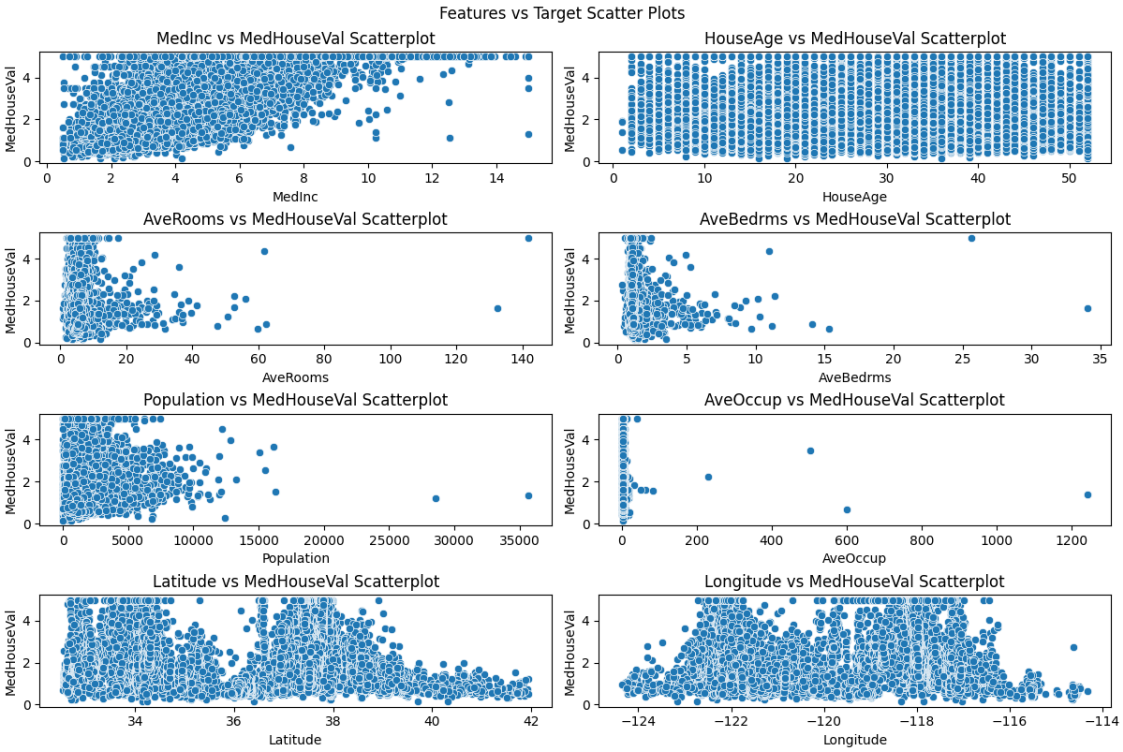
1.2.3 Target distribution

[6]:
```
eda.plot_target_distribution()
plt.show()
```



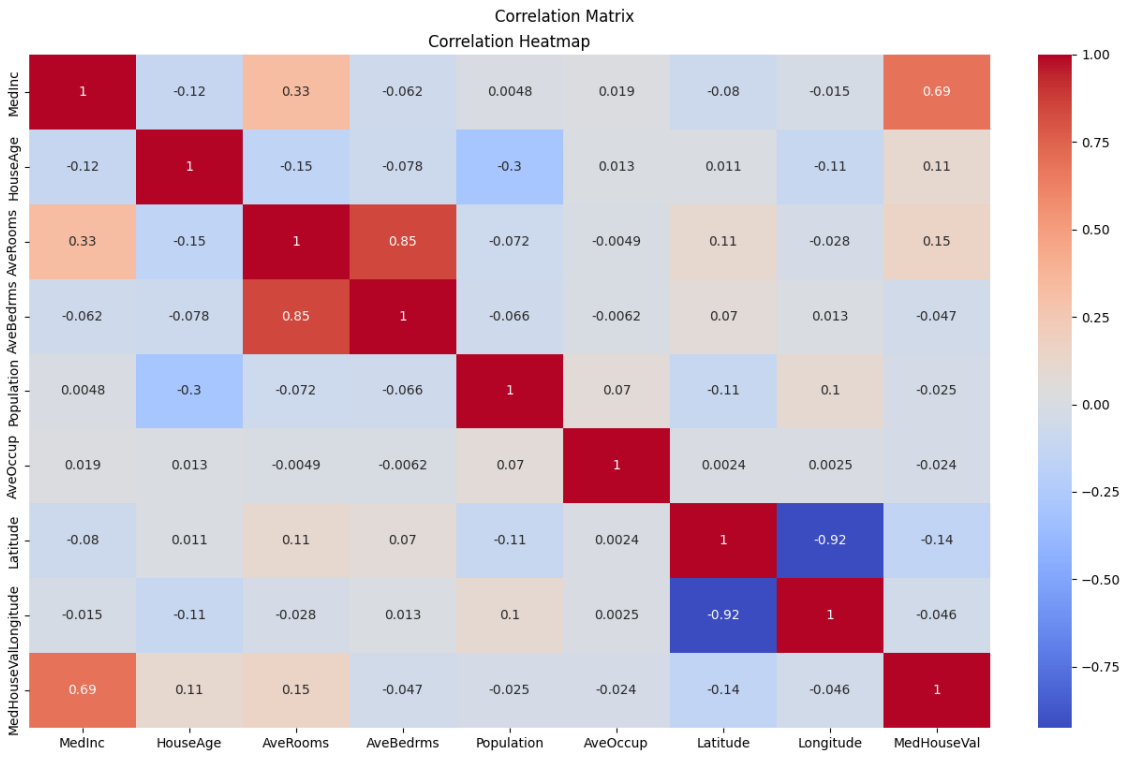
1.2.4 Scatter

[7]:
```
eda.plot_scatter_features_target()
plt.show()
```



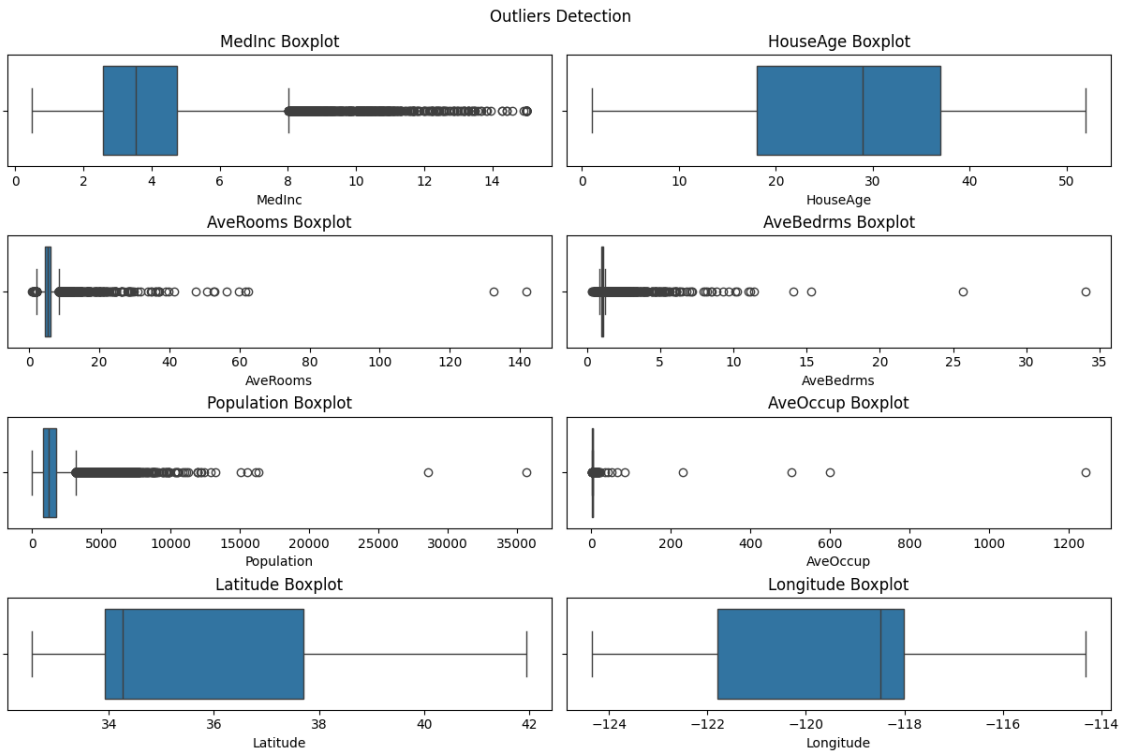
1.2.5 Correlation matrix

[4]:
```
eda.plot_corrrelation_matrix()
plt.show()
```



1.2.6 Outliers

[9]:
```
eda.plot_outliers()
plt.show()
```



1.3 Models

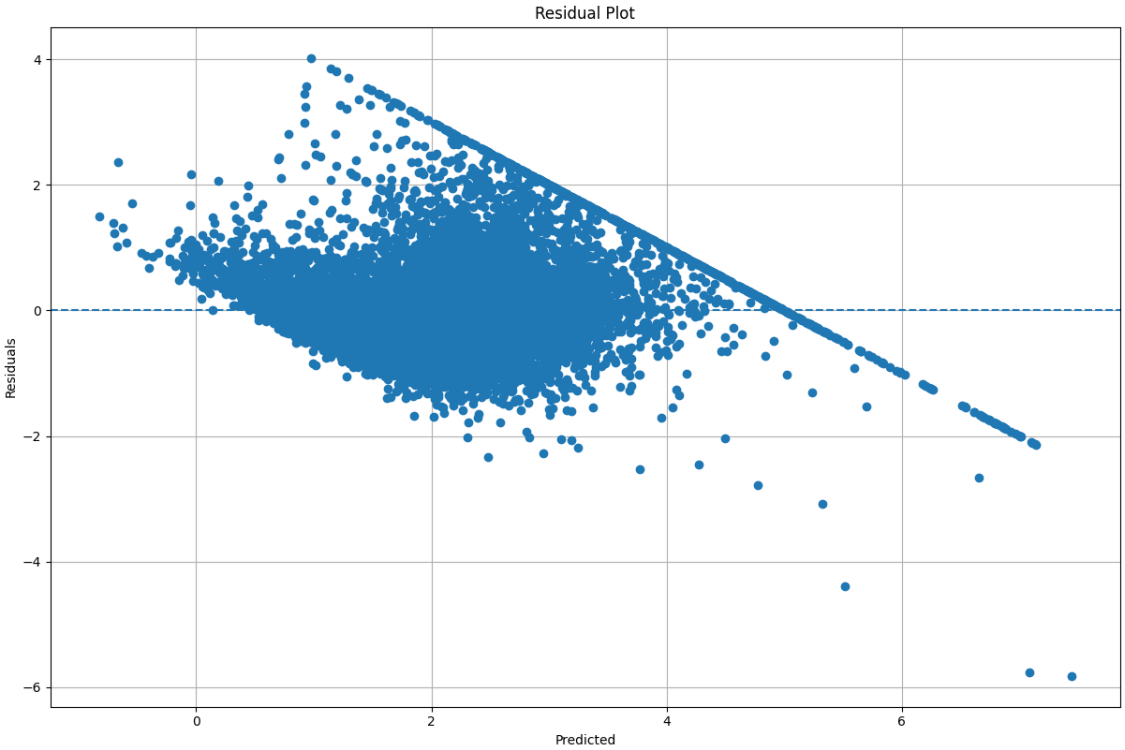
1.3.1 Linear regression

Gauss-Markov assumption

[10]:
```
model = get_pipeline(OLS())
model.fit(d.X_train, d.y_train)
y_pred_train = model.predict(d.X_train)
diag = GaussMarkovDiagnostics()
```

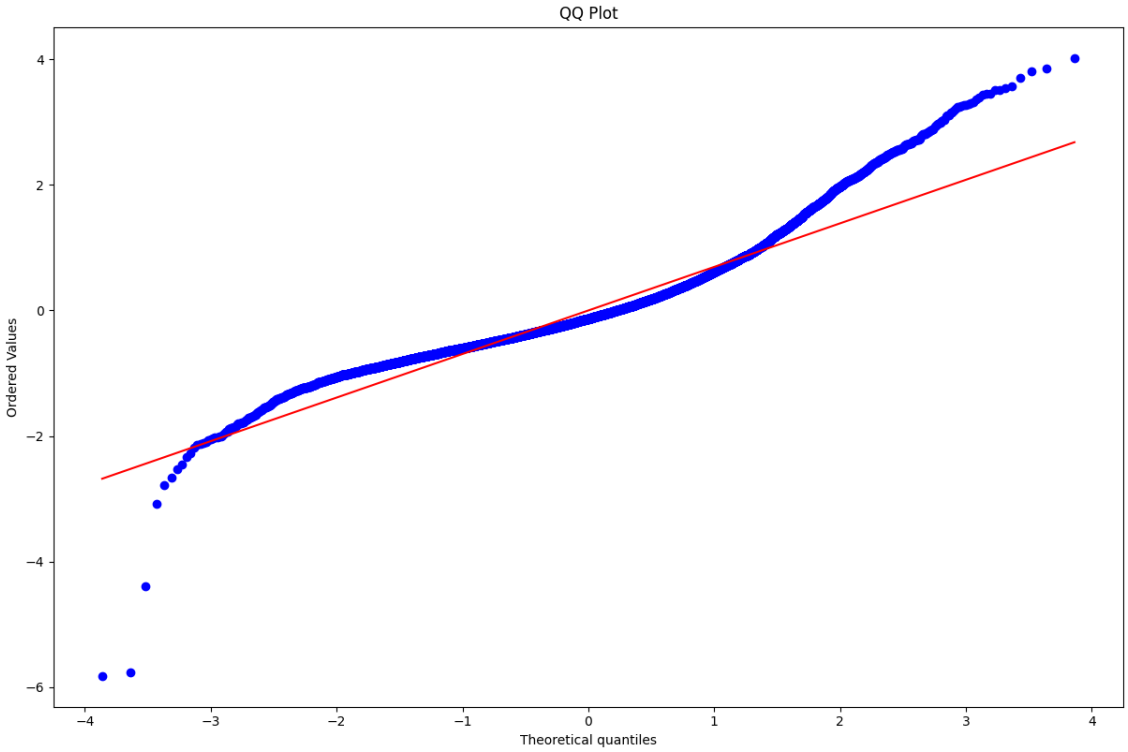
Residuals plot

[11]:
```
residuals_train = d.y_train - y_pred_train
diag.plot_residuals(d.y_train, y_pred_train)

plt.show()
```



QQ-plot

[12]:
```
diag.plot_qq(residuals_train)
plt.show()
```



Breusch–Pagan test

[13]:
```
result = diag.breusch_pagan_test(d.X_train, residuals_train)
display(result)
```

```
{'LM Stat': np.float64(1248.615593028101),
 'LM p-value': np.float64(2.9971092972062764e-264),
 'F Stat': np.float64(173.4517799184423),
 'F p-value': np.float64(1.049113595091813e-278)}
```

1.3.2 Regularization and features selecton

Ridge Regression

[5]:
```
lambdas = np.logspace(-10, 1, 12)
```

[6]:
```
model = get_pipeline(RidgeRegressionCV(alphas=lambdas, cv=10))
model.fit(d.X_train, d.y_train)
```

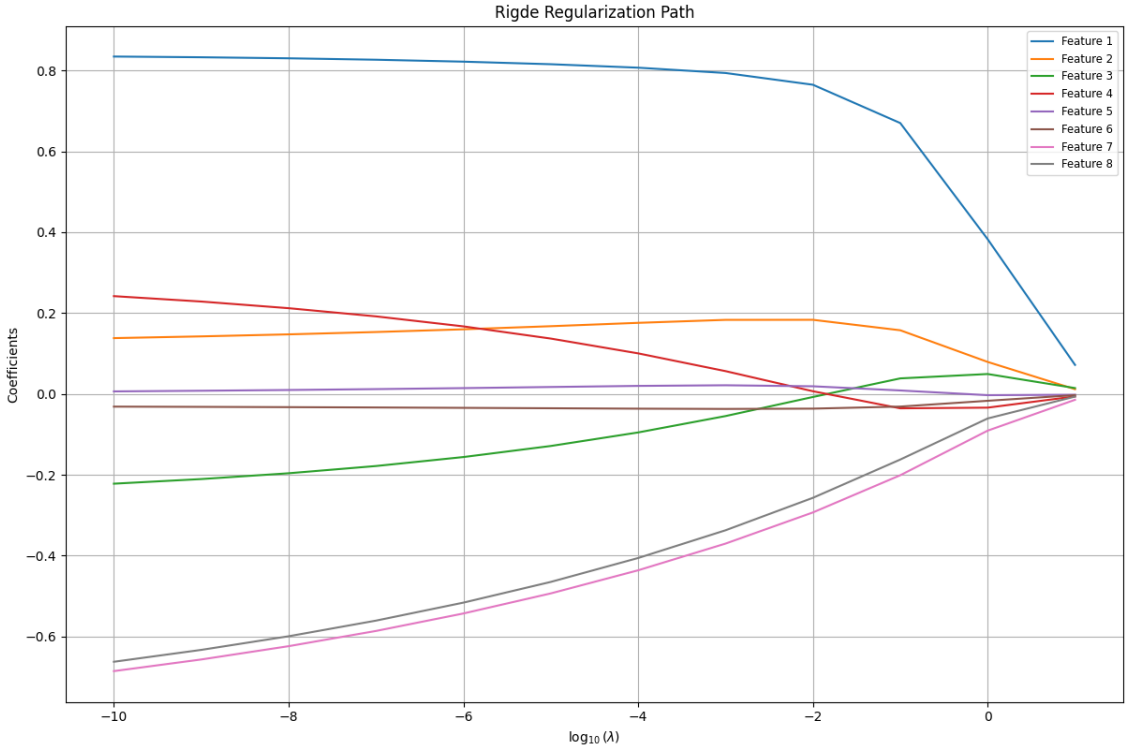
[6]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('predictor',
                 RidgeRegressionCV(alphas=[np.float64(10.0), np.float64(1.0),
                                           np.float64(0.1), np.float64(0.01),
                                           np.float64(0.001),
                                           np.float64(0.0001),
                                           np.float64(9.999999999999999e-06),
```

```
                                        np.float64(1e-06), np.float64(1e-07),
                                        np.float64(1e-08), np.float64(1e-09),
                                        np.float64(1e-10)]))])
```

[16]:
```
print(model.named_steps["predictor"].best_alpha_)
```

```
1e-10
```

[17]:
```
model.named_steps["predictor"].plot_regularization_path(title="Rigde␣
↪Regularization Path")
```



Lasso Regression

[7]:
```
lambdas = np.logspace(-10, 1, 12)
```

[8]:
```
model = get_pipeline(LassoRegressionCV(alphas=lambdas, cv=10))
model.fit(d.X_train, d.y_train)
```

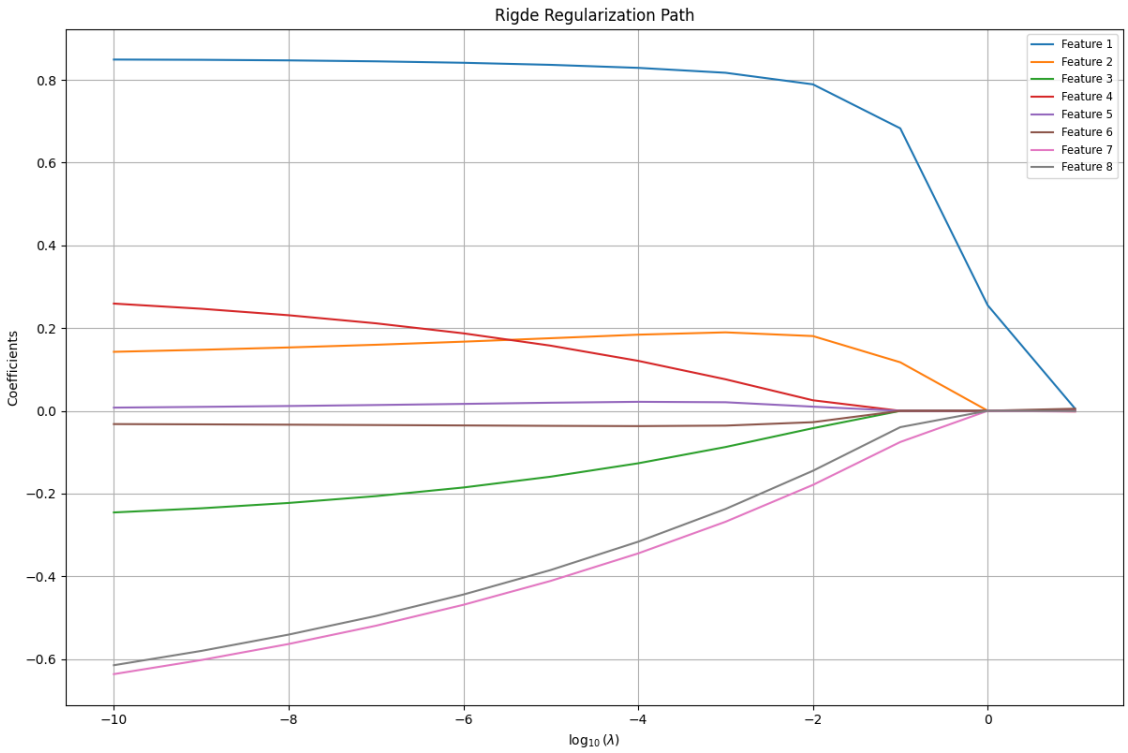
[8]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('predictor',
                 LassoRegressionCV(alphas=[np.float64(10.0), np.float64(1.0),
                                           np.float64(0.1), np.float64(0.01),
                                           np.float64(0.001),
                                           np.float64(0.0001),
```

```
                                        np.float64(9.999999999999999e-06),
                                        np.float64(1e-06), np.float64(1e-07),
                                        np.float64(1e-08), np.float64(1e-09),
                                        np.float64(1e-10)]))])
```

[20]:
```
print(model.named_steps["predictor"].best_alpha_)
```

```
1e-10
```

[21]:
```
model.named_steps["predictor"].plot_regularization_path(title="Rigde␣
↪Regularization Path")
```



Elastic Net

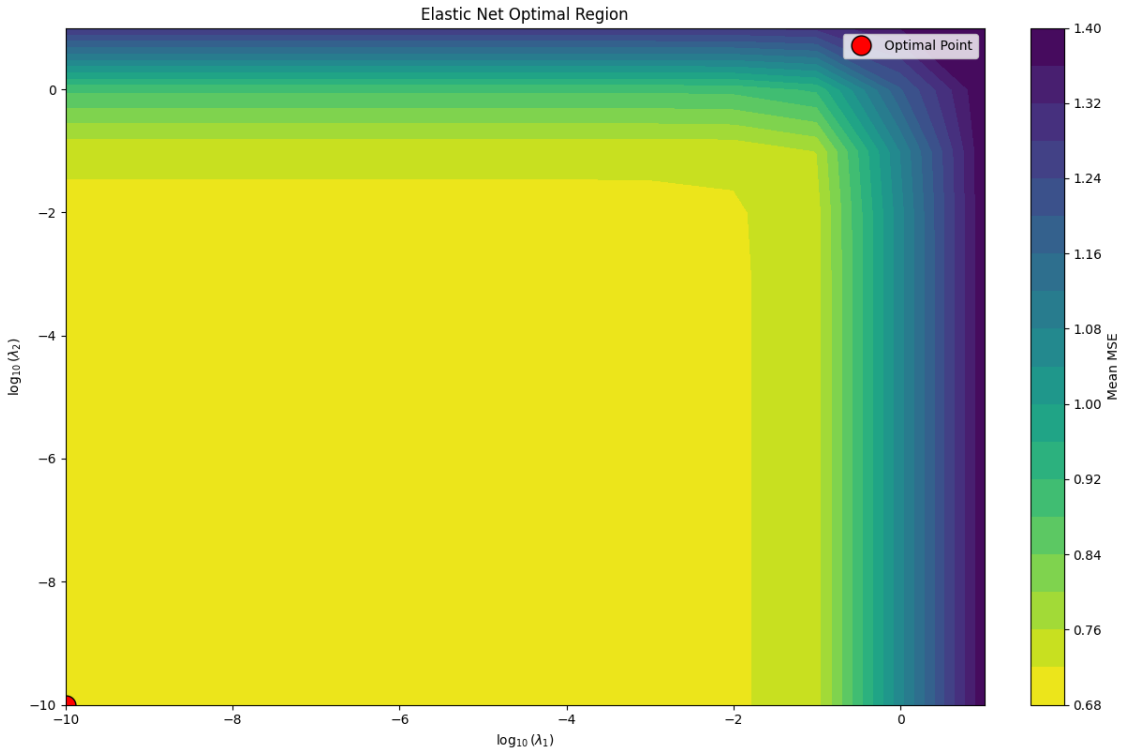
[22]:
```
lambda_1s = np.logspace(-10, 1, 12)
lambda_2s = np.logspace(-10, 1, 12)
```

[23]:
```
model = get_pipeline(ElasticNetCV(alpha_1s=lambda_1s, alpha_2s=lambda_2s,␣
↪cv=10))
model.fit(d.X_train, d.y_train)
```

[23]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('predictor',
                 ElasticNetCV(alpha_1s=[np.float64(10.0), np.float64(1.0),
```

```
                                    np.float64(0.1), np.float64(0.01),
                                    np.float64(0.001), np.float64(0.0001),
                                    np.float64(9.999999999999999e-06),
                                    np.float64(1e-06), np.float64(1e-07),
                                    np.float64(1e-08), np.float64(1e-09),
                                    np.float64(1e-10)],
                           alpha_2s=[np.float64(10.0), np.float64(1.0),
                                    np.float64(0.1), np.float64(0.01),
                                    np.float64(0.001), np.float64(0.0001),
                                    np.float64(9.999999999999999e-06),
                                    np.float64(1e-06), np.float64(1e-07),
                                    np.float64(1e-08), np.float64(1e-09),
                                    np.float64(1e-10)]))])
```

[24]:
```
model.named_steps["predictor"].plot_optimal_region(title="Elastic Net Optimal␣
↪Region")
```



Features Selection

[25]:
```
k_features = 'auto'
```

[14]:
```
ols_model = get_pipeline(OLS())
lasso_model = get_pipeline(Lasso())
```

Forward

[15]:
```
forward_selection(d.X_train, d.y_train, ols_model, k_features, d.feature_names)
```

[15]:
```
['MedInc', 'HouseAge', 'Latitude', 'Longitude']
```

[16]:
```
forward_selection(d.X_train, d.y_train, lasso_model, k_features, d.
↪feature_names)
```

[16]:
```
['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms']
```

Backward

[17]:
```
backward_elimination(d.X_train, d.y_train, ols_model, k_features, d.
↪feature_names)
```

[17]:
```
['MedInc', 'HouseAge', 'Latitude', 'Longitude']
```

[18]:
```
backward_elimination(d.X_train, d.y_train, lasso_model, k_features, d.
↪feature_names)
```

[18]:
```
['Population', 'AveOccup', 'Latitude', 'Longitude']
```

Lasso

[19]:
```
lambdas = np.logspace(-10, 1, 12)
```

[20]:
```
lasso_selection(d.X_train, d.y_train, d.feature_names, alphas=lambdas, cv=10)
```

[20]:
```
['MedInc',
 'HouseAge',
 'AveRooms',
 'AveBedrms',
 'AveOccup',
 'Latitude',
 'Longitude']
```

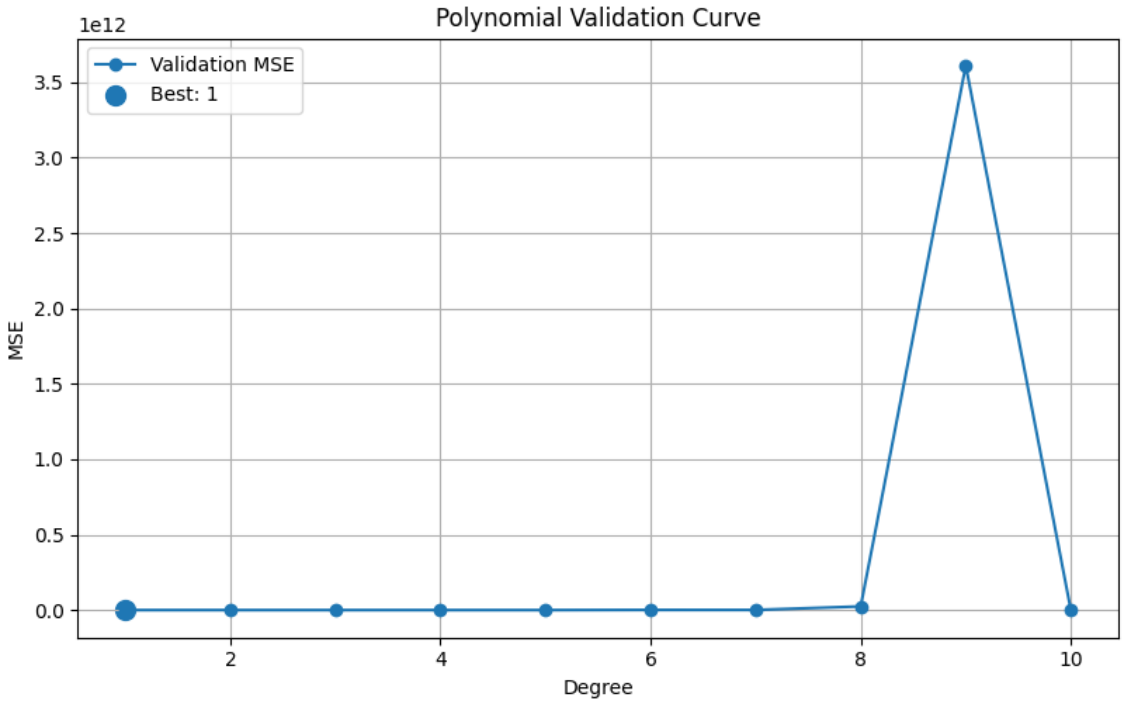
1.3.3 Non-linear basis and Ablation study

Validation Curve

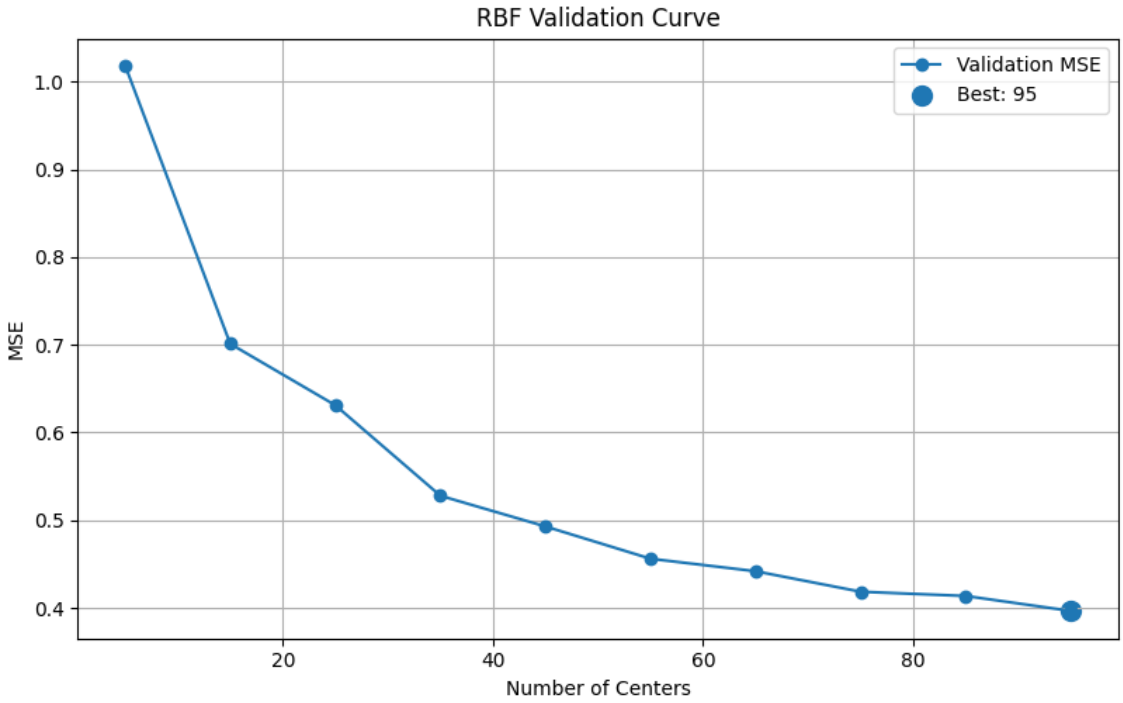
[2]:
```
exp = ValidationExperiment()
exp.load_data()
exp.run_all()
```

```
degree=1 → MSE=0.5349
degree=2 → MSE=2.0967
degree=3 → MSE=65.1358
degree=4 → MSE=985.9195
degree=5 → MSE=642335.1990
degree=6 → MSE=990824603.3006
degree=7 → MSE=870914361.8040
degree=8 → MSE=24147262204.6357
```

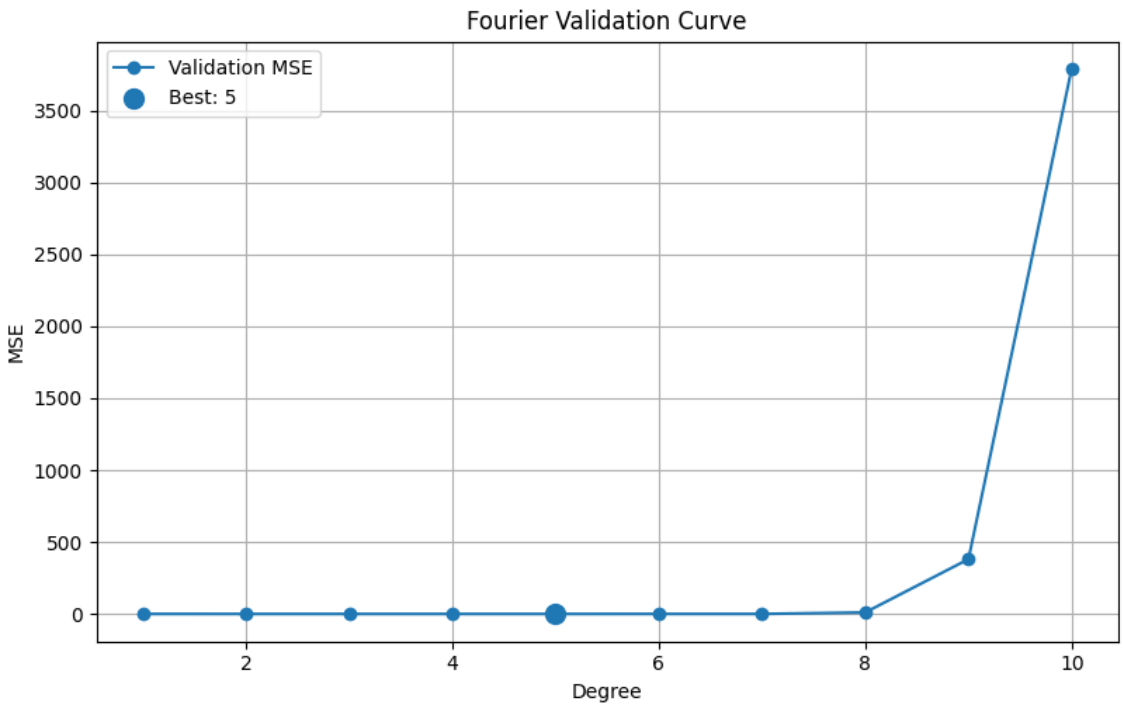
```
degree=9 → MSE=3611468799507.6123
degree=10 → MSE=7751274.7854
```




```
n_centers=5 → MSE=1.0185
n_centers=15 → MSE=0.7012
n_centers=25 → MSE=0.6311
n_centers=35 → MSE=0.5279
n_centers=45 → MSE=0.4929
n_centers=55 → MSE=0.4561
n_centers=65 → MSE=0.4418
n_centers=75 → MSE=0.4185
n_centers=85 → MSE=0.4136
n_centers=95 → MSE=0.3966
```




```
n_terms=1 → MSE=0.5488
n_terms=2 → MSE=0.4482
n_terms=3 → MSE=0.4356
n_terms=4 → MSE=0.4202
n_terms=5 → MSE=0.4199
n_terms=6 → MSE=0.5948
n_terms=7 → MSE=0.6172
n_terms=8 → MSE=10.8597
n_terms=9 → MSE=381.1882
n_terms=10 → MSE=3791.8363
```

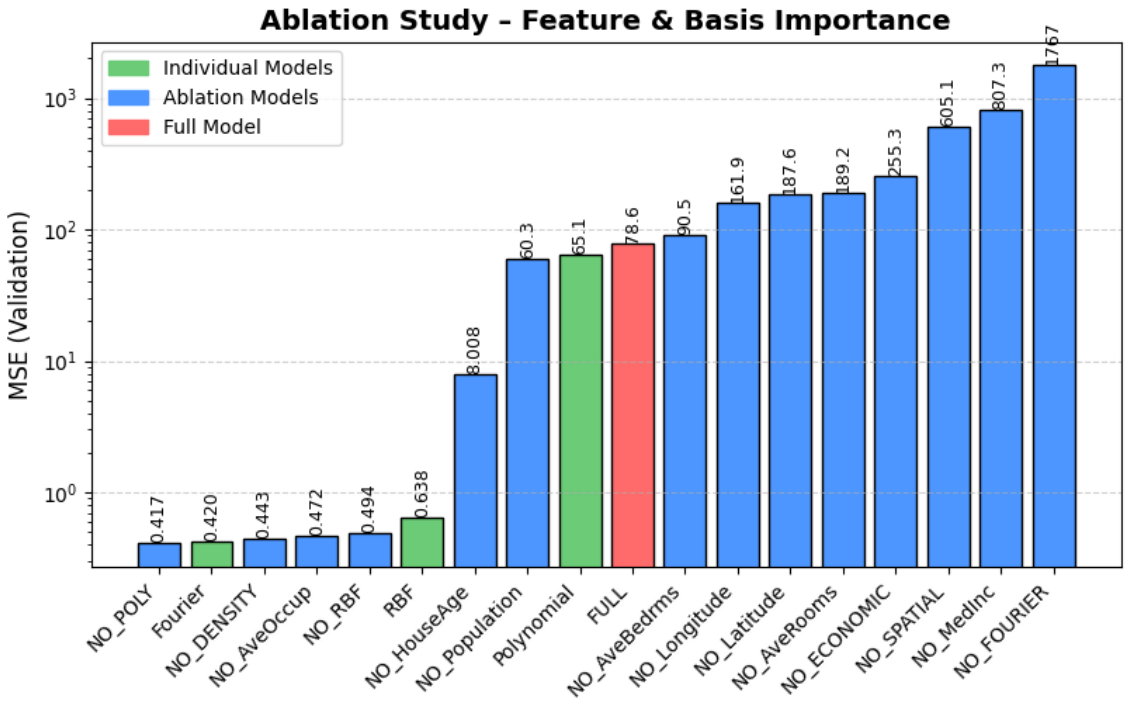


Ablation Study

[2]:
```
exp = AblationStudy()
exp.run_all()
exp.print_results()
exp.plot()
```

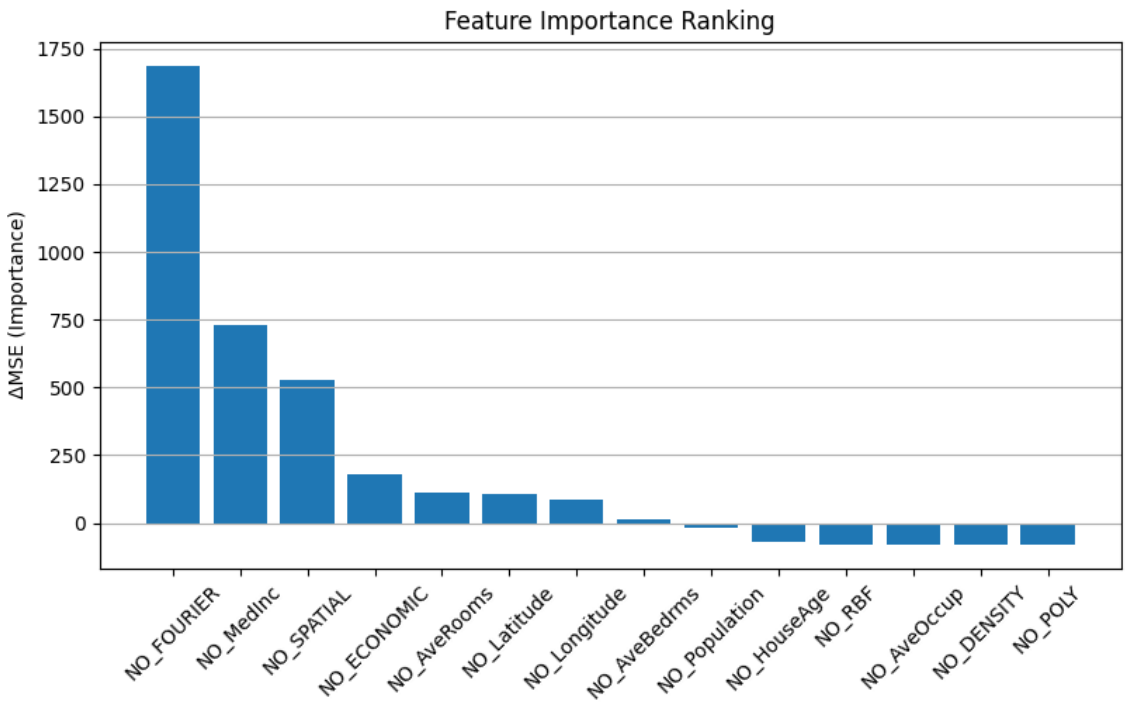
```
===== VALIDATION ABLATION RESULTS =====
RBF                   : 0.637879
Fourier               : 0.419886
Polynomial            : 65.135821
NO_RBF                : 0.493888
NO_FOURIER            : 1767.324896
NO_POLY               : 0.417499
NO_ECONOMIC           : 255.325352
NO_DENSITY            : 0.442693
NO_SPATIAL            : 605.146135
NO_MedInc             : 807.261046
NO_HouseAge           : 8.008103
NO_AveRooms           : 189.150772
NO_AveBedrms          : 90.545996
NO_Population         : 60.263111
NO_AveOccup           : 0.471775
NO_Latitude           : 187.599768
```

```
NO_Longitude          : 161.883225
FULL                  : 78.553132
```



[3]:
```
exp.rank_features()
exp.plot_importance()
```

```
===== FEATURE RANKING (ΔMSE) =====
NO_FOURIER            : 1688.771764
NO_MedInc             : 728.707915
NO_SPATIAL            : 526.593003
NO_ECONOMIC           : 176.772220
NO_AveRooms           : 110.597641
NO_Latitude           : 109.046636
NO_Longitude          : 83.330093
NO_AveBedrms          : 11.992864
NO_Population         : -18.290021
NO_HouseAge           : -70.545029
NO_RBF                : -78.059244
NO_AveOccup           : -78.081357
NO_DENSITY            : -78.110439
NO_POLY               : -78.135632
```



Interaction

[12]:
```
X_train, y_train = d.X_train, d.y_train
X_val, y_val = d.X_val, d.y_val

y_train = y_train.ravel()
y_val = y_val.ravel()

# ===== MODEL WITH INTERACTION =====
model_inter = get_pipeline(
    predictor=LinearRegression(),
    features=None,
    use_interaction=True
)

model_inter.fit(X_train, y_train)
y_pred_inter = model_inter.predict(X_val)

mse_inter = mean_squared_error(y_val, y_pred_inter)
print("MSE (with interaction):", mse_inter)
```

```
MSE (with interaction): 4.012430981538565
```

[13]:
```
model_base = get_pipeline(
    predictor=LinearRegression(),
    features=PolynomialBasis(degree=2),
```

```
    use_interaction=False
)

model_base.fit(X_train, y_train)
y_pred_base = model_base.predict(X_val)

mse_base = mean_squared_error(y_val, y_pred_base)
print("MSE (no interaction):", mse_base)

print("Improvement:", mse_base - mse_inter)
```

```
MSE (no interaction): 2.096747042719358
Improvement: -1.9156839388192073
```

1.4 Advanced

1.4.1 Bayesian Regression

[7]:
```
# ===== MODEL =====
X_train, y_train = d.X_train, d.y_train
X_test, y_test = d.X_test, d.y_test
model = BayesianLinearRegression(alpha=1.0, beta=10.0)
model.fit(X_train, y_train)
```

[7]:
```
BayesianLinearRegression()
```

[8]:
```
 # ===== POSTERIOR =====
m_N, S_N = model.get_posterior()
print("Posterior mean shape:", m_N.shape)
print("Posterior covariance shape:", S_N.shape)
```

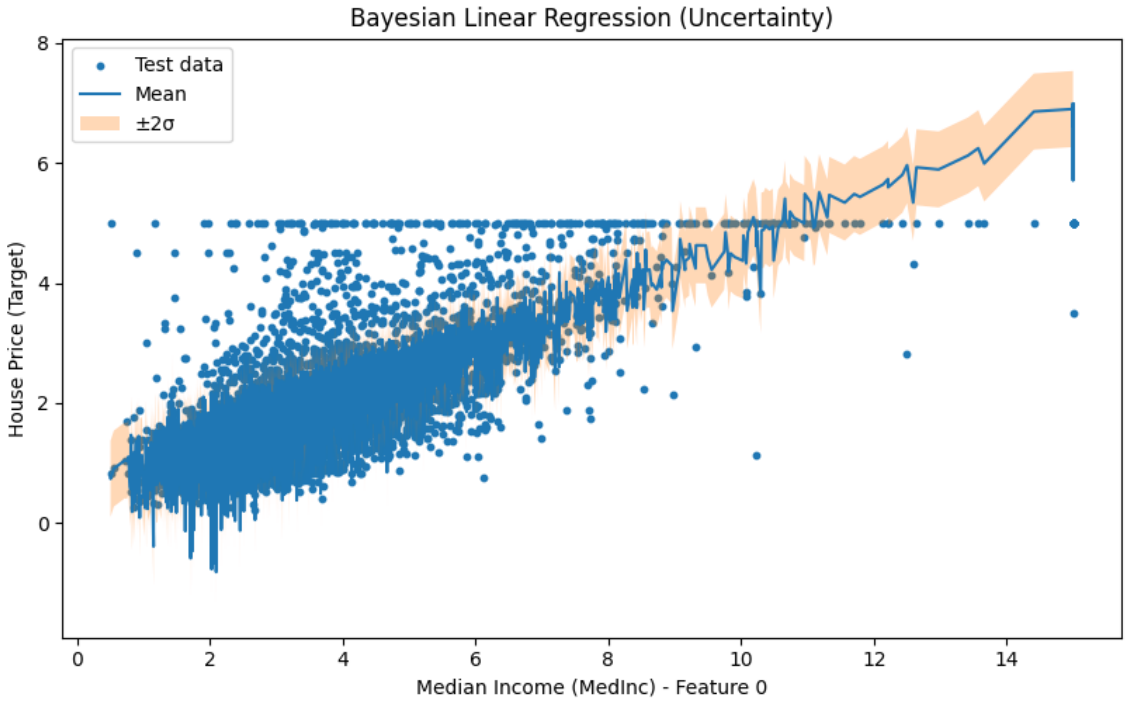
```
Posterior mean shape: (9, 1)
Posterior covariance shape: (9, 9)
```

[9]:
```
 # ===== PREDICT =====
y_mean, y_std = model.predict_dist(X_test)
mse = np.mean((y_test - y_mean) ** 2)
print(f"MSE = {mse:.6f}")
```

```
MSE = 0.524602
```

[10]:
```
# ===== PLOT (1D visualization) =====
X_plot = X_test[:, 0]
idx = np.argsort(X_plot)

plt.figure(figsize=(8, 5))

# data test
plt.scatter(X_plot, y_test, s=10, label="Test data")
```

```
# mean prediction
plt.plot(X_plot[idx], y_mean[idx], label="Mean")

# vùng bất định ±2
plt.fill_between(
    X_plot[idx],
    y_mean[idx] - 2*y_std[idx],
    y_mean[idx] + 2*y_std[idx],
    alpha=0.3,
    label="±2 "
)
plt.xlabel("Median Income (MedInc) - Feature 0")
plt.ylabel("House Price (Target)")
plt.legend()
plt.title("Bayesian Linear Regression (Uncertainty)")
plt.show()
```



[11]:
```
#Optimazation alpha , beta using Evidence Maximization to compare with CV
start = time.time()

model_em = BayesianLinearRegression()
model_em.evidence_maximization(X_train, y_train)
```

```python
y_pred_em = model_em.predict(X_test)

em_time = time.time() - start
em_mse = mean_squared_error(y_test, y_pred_em)

#  CV
param_grid = {
    "alpha": [0.1, 1, 10],
    "beta": [1, 10, 100]
}

start = time.time()

grid = GridSearchCV(
    BayesianLinearRegression(),
    param_grid,
    cv=3
)
grid.fit(X_train, y_train)

y_pred_cv = grid.predict(X_test)

cv_time = time.time() - start
cv_mse = mean_squared_error(y_test, y_pred_cv)

# ================= RESULT =================
print("\n===== EVIDENCE MAXIMIZATION =====")
print("alpha:", model_em.alpha)
print("beta:", model_em.beta)
print("MSE:", em_mse)
print("Time:", em_time)

print("\n===== CROSS VALIDATION =====")
print("Best params:", grid.best_params_)
print("MSE:", cv_mse)
print("Time:", cv_time)

print("\n===== COMPARISON =====")
print(f"EM faster than CV? {em_time < cv_time}")
print(f"EM better MSE than CV? {em_mse < cv_mse}")
```

```
===== EVIDENCE MAXIMIZATION =====
alpha: 0.0064293529731089686
beta: 1.9166775717920053
MSE: 0.5252559397214996
Time: 0.05368614196777344
```

```
===== CROSS VALIDATION =====
Best params: {'alpha': 0.1, 'beta': 100}
MSE: 0.5253081487282695
Time: 0.06824374198913574

===== COMPARISON =====
EM faster than CV? True
EM better MSE than CV? True
```

Kernel Ridge Regression

[15]:
```
models = {
    "Ridge": get_pipeline(RidgeRegression()),
    "RBF Ridge": get_pipeline(KernelRidgeCV(kernel='rbf', cv=2)),
    "Polynomial Ridge": get_pipeline(KernelRidgeCV(kernel='polynomial', cv=2))
}
evaluator = Evaluator()

result = evaluator.compare_models_cv(models, d.X_train, d.y_train)

print(result)

result = evaluator.compare_models_test(models, d.X_train, d.y_train,
                                       d.X_test, d.y_test)
print(result)

stat_result = evaluator.compare_models_statistical(
    models,
    d.X_train,
    d.y_train
)

print(stat_result)
```

```
              Model                MSE              RMSE               MAE  \
0             Ridge   0.8875 ± 0.0352  0.9419 ± 0.0186  0.6829 ± 0.0163
1         RBF Ridge   0.3583 ± 0.0275  0.5982 ± 0.0224  0.4185 ± 0.0098
2  Polynomial Ridge  4.2643 ± 10.9304  1.2859 ± 1.6158  0.5499 ± 0.2425

                 R2
0   0.3288 ± 0.0244
1   0.7289 ± 0.0210
2  -2.4059 ± 8.7993
              Model       MSE      RMSE       MAE        R2
0             Ridge  0.895142  0.946120  0.680719  0.335008
1         RBF Ridge  0.357096  0.597575  0.417524  0.734717
2  Polynomial Ridge  0.434008  0.658793  0.466575  0.677580
     Model A           Model B  p-value (t-test)  p-value (Wilcoxon)
0      Ridge         RBF Ridge      1.998716e-13            0.001953
```

```
1      Ridge  Polynomial Ridge      3.789153e-01            0.431641
2  RBF Ridge  Polynomial Ridge      3.117077e-01            0.001953
```

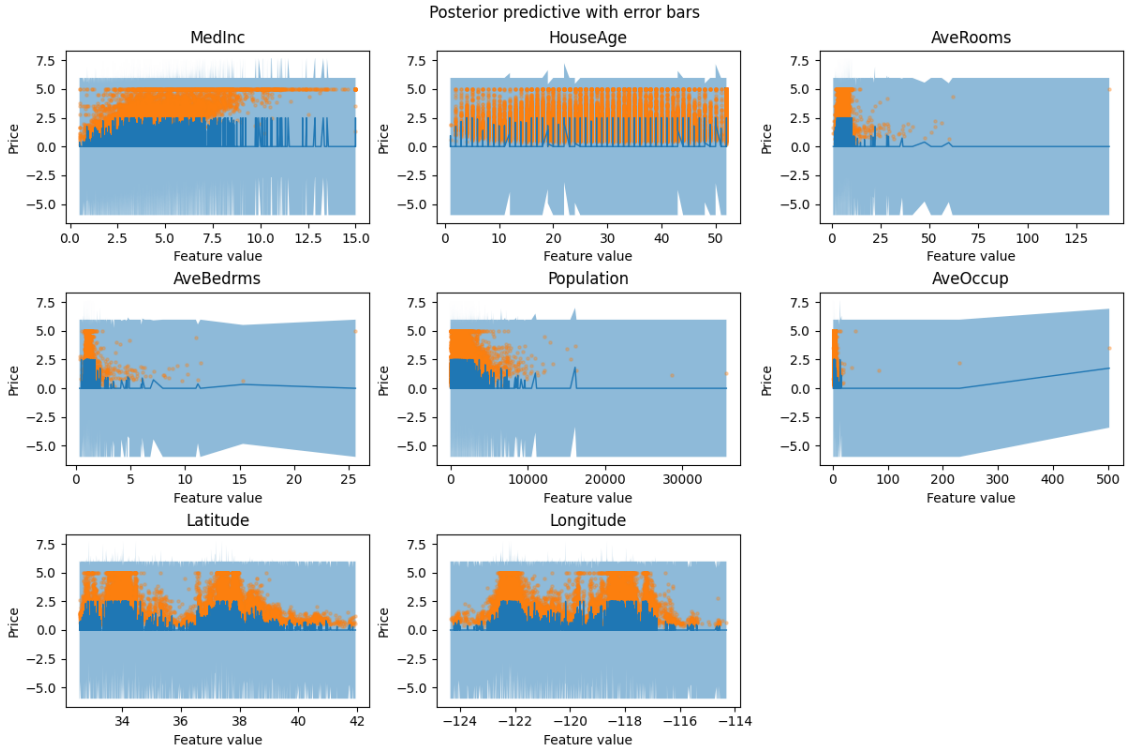
1.4.2 Gaussian Process Regression

[24]:
```
rng = np.random.default_rng(42)
idx = rng.choice(len(d.X_train), 2000, replace=False)
X_sub = d.X_train[idx]
y_sub = d.y_train[idx]

model = get_pipeline(GPR())
model.fit(X_sub, y_sub)

y_pred, y_std = model.predict(d.X_train)
y_true = d.y_train
print("min std:", y_std.min())
print("max std:", y_std.max())
```

```
/home/quihoang/Projects/supervised-learning-project/.venv/lib64/python3.14/site-
packages/sklearn/gaussian_process/kernels.py:440: ConvergenceWarning: The
optimal value found for dimension 0 of parameter k1__k2__length_scale is close
to the specified lower bound 1e-05. Decreasing the bound and calling fit again
may find a better value.
  warnings.warn(

min std: 2.6375261718730796
max std: 3.0450061851458416
```

[37]:
```
feature_names = getattr(d, "feature_names", [f"Feature {i}" for i in range(d.X.
↪shape[1])])

n_features = d.X.shape[1]

fig, axes = plt.subplots(3, 3)
axes = axes.flatten()
for i in range(n_features):
    x_plot = d.X_train[:, i]
    idx = np.argsort(x_plot)

    axes[i].plot(x_plot[idx], y_pred[idx], label="Pred", linewidth=1)

    # Uncertainty band
    axes[i].fill_between(
        x_plot[idx],
        y_pred[idx] - 1.96 * y_std[idx],
        y_pred[idx] + 1.96 * y_std[idx],
        alpha=0.5
    )
```

```
    # True points
    axes[i].scatter(x_plot, y_true, s=5, alpha=0.3)

    axes[i].set_title(feature_names[i])
    axes[i].set_xlabel("Feature value")
    axes[i].set_ylabel("Price")

axes[8].axis("off")
fig.suptitle("Posterior predictive with error bars")
plt.show()
```



1.4.3 Robust Regression

[18]:
```
huber = get_pipeline(IRLS(loss="huber"))
huber.fit(d.X_train, d.y_train)
y_pred_huber = huber.predict(d.X_test)

student = get_pipeline(IRLS(loss="student-t"))
student.fit(d.X_train, d.y_train)
y_pred_student = student.predict(d.X_test)

model_ols = get_pipeline(OLS())
```

```
model_ols.fit(d.X_train, d.y_train)
y_pred_ols = model_ols.predict(d.X_test)

y_true = d.y_test

print("=== Overall MSE ===")
print(f"OLS: {mean_squared_error(y_true, y_pred_ols):.4f}")
print(f"Huber IRLS: {mean_squared_error(y_true, y_pred_huber):.4f}")
print(f"Student-t IRLS: {mean_squared_error(y_true, y_pred_student):.4f}")
```

```
=== Overall MSE ===
OLS: 0.5253
Huber IRLS: 0.5180
Student-t IRLS: 0.4726
```

[19]:
```
def compute_metrics(y_true, y_pred):
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    return mse, rmse, mae, r2


def outlier_sensitivity_df(models, data, outlier_ratio=0.1, noise_scale=5.0):
    rows = []

    # ===== CLEAN TRAIN =====
    clean_scores = {}

    for name, model in models.items():
        model.fit(data.X_train, data.y_train)
        y_pred = model.predict(data.X_test)

        mse, rmse, mae, r2 = compute_metrics(data.y_test, y_pred)
        clean_scores[name] = (mse, rmse, mae, r2)

    # ===== ADD OUTLIERS =====
    y_train_noisy = data.y_train.copy()

    idx = np.random.choice(
        len(y_train_noisy),
        int(outlier_ratio * len(y_train_noisy)),
        replace=False
    )

    y_train_noisy[idx] += noise_scale * np.std(y_train_noisy)
```

```
    # ===== NOISY TRAIN =====
    noisy_scores = {}

    for name, model in models.items():
        model.fit(data.X_train, y_train_noisy)
        y_pred = model.predict(data.X_test)

        mse, rmse, mae, r2 = compute_metrics(data.y_test, y_pred)
        noisy_scores[name] = (mse, rmse, mae, r2)

    # ===== BUILD DATAFRAME =====
    for name in models.keys():
        cmse, crmse, cmae, cr2 = clean_scores[name]
        nmse, nrmse, nmae, nr2 = noisy_scores[name]

        rows.append({
            "Model": name,

            "MSE_clean": cmse,
            "MSE_noisy": nmse,
            "MSE_sensitivity": (nmse - cmse) / (cmse + 1e-8),

            "RMSE_clean": crmse,
            "RMSE_noisy": nrmse,
            "RMSE_sensitivity": (nrmse - crmse) / (crmse + 1e-8),

            "MAE_clean": cmae,
            "MAE_noisy": nmae,
            "MAE_sensitivity": (nmae - cmae) / (cmae + 1e-8),

            "R2_clean": cr2,
            "R2_noisy": nr2,
            "R2_sensitivity": (cr2 - nr2),
        })

    return pd.DataFrame(rows)

models = {
    "OLS": get_pipeline(OLS()),
    "IRLS (Huber)": get_pipeline(IRLS(loss="huber")),
    "IRLS (Student-t)": get_pipeline(IRLS(loss="student-t")),
}

df = outlier_sensitivity_df(models, d)
# df.round(4).to_csv("regression-irls-outliers-sensitivity.csv", index=False)
```

```
display(df)
```

```
              Model  MSE_clean  MSE_noisy  MSE_sensitivity  RMSE_clean  \
0               OLS   0.525331   0.861938         0.640752    0.724797
1      IRLS (Huber)   0.518014   0.525928         0.015279    0.719732
2  IRLS (Student-t)   0.472567   0.474802         0.004730    0.687435

   RMSE_noisy  RMSE_sensitivity  MAE_clean  MAE_noisy  MAE_sensitivity  \
0    0.928406          0.280918   0.529916   0.796649         0.503348
1    0.725209          0.007610   0.512925   0.539434         0.051682
2    0.689059          0.002362   0.490149   0.504871         0.030036

   R2_clean  R2_noisy  R2_sensitivity
0  0.609737  0.359676        0.250062
1  0.615173  0.609293        0.005880
2  0.648935  0.647275        0.001661
```

1.4.4 Bias-Variance Analysis

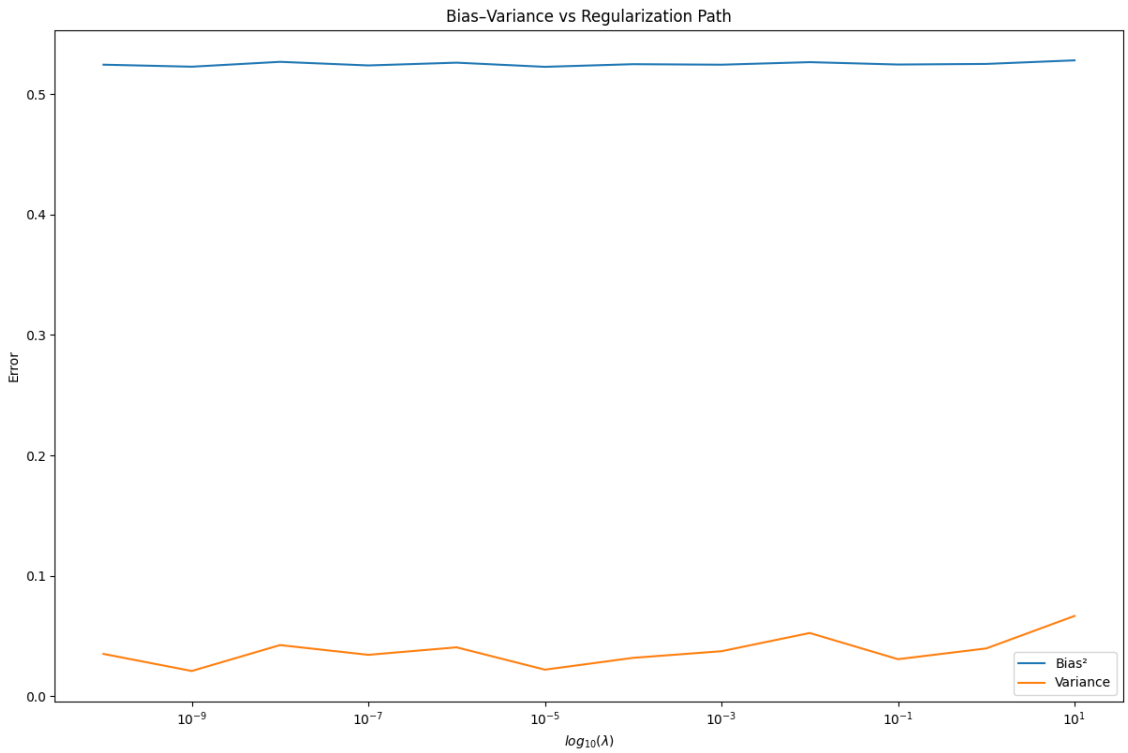
Ridge

[20]:
```
model = get_pipeline(BiasVarianceAnalyzer(estimator=Ridge()))
model.fit(d.X_train, d.y_train)
```

[20]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('predictor', BiasVarianceAnalyzer(estimator=Ridge()))])
```

[21]:
```
model.named_steps["predictor"].plot()
```

[21]:
```
(<Figure size 1200x800 with 1 Axes>,
 <Axes: title={'center': 'Bias-Variance vs Regularization Path'},
xlabel='$log_{10}(\\lambda)$', ylabel='Error'>)
```

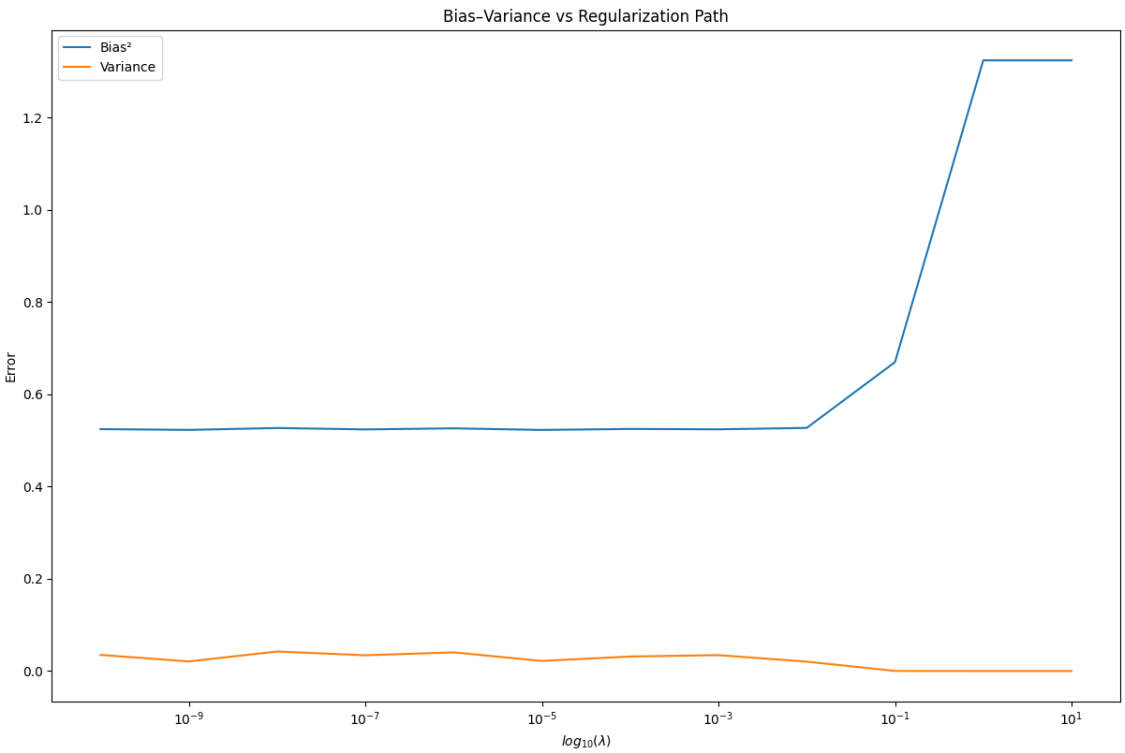


[3]:
```
model = get_pipeline(BiasVarianceAnalyzer(estimator=Lasso()))
model.fit(d.X_train, d.y_train)
```

[3]:
```
Pipeline(steps=[('scaler', StandardScaler()),
                ('predictor', BiasVarianceAnalyzer(estimator=Lasso()))])
```

[4]:
```
model.named_steps["predictor"].plot()
```

[4]:
```
(<Figure size 1200x800 with 1 Axes>,
 <Axes: title={'center': 'Bias-Variance vs Regularization Path'},
xlabel='$log_{10}(\\lambda)$', ylabel='Error'>)
```



1.5 Evaluation

[3]:
```
logger = get_logger("logger", "log.txt")
# logger = None
```

[4]:
```
models = {
    "OLS": get_pipeline(OLS()),
    "MBGD": get_pipeline(MBGD(lr_sched="step_decay")),
    "WLS": get_pipeline(WLS()),
    "Ridge": get_pipeline(get_pipeline(RidgeRegressionCV()).fit(d.X_train, d.
 ↪y_train).named_steps["predictor"].final_model_),
    "Lasso": get_pipeline(get_pipeline(LassoRegressionCV()).fit(d.X_train, d.
 ↪y_train).named_steps["predictor"].final_model_),
    "Elastic Net": get_pipeline(get_pipeline(ElasticNetCV()).fit(d.X_train, d.
 ↪y_train).named_steps["predictor"].final_model_),
    "Polynomial": get_pipeline(OLS(), PolynomialBasis(degree=2)),
    "RBF": get_pipeline(OLS(), RBF(n_centers=10, gamma=0.1)),
    "Fourier": get_pipeline(OLS(), FourierBasis(n_terms=5))
}
```

1.5.1 Evaluate on test set, cross-validation, t-test, Wilcoxon signed-rank test

[5]:
```
evaluator = Evaluator(logger=logger, log_model=True)
```

On test set

[6]:
```
result = evaluator.compare_models_test(models, d.X_train, d.y_train,
                                       d.X_test, d.y_test)
# result.round(4).to_csv("regression-models-test.csv", index=False)
print("===Test Result===")
display(result)
```

```
===Test Result===
```

	Model	MSE	RMSE	MAE	R2
0	OLS	0.525331	0.724797	0.529916	0.609737
1	MBGD	0.525415	0.724855	0.529913	0.609674
2	WLS	0.527603	0.726363	0.520482	0.608049
3	Ridge	0.729670	0.854207	0.595350	0.457936
4	Lasso	0.747721	0.864709	0.605148	0.444526
5	Elastic Net	0.766020	0.875225	0.615269	0.430932
6	Polynomial	0.494023	0.702868	0.514549	0.632996
7	RBF	0.697130	0.834943	0.627836	0.482110
8	Fourier	0.387296	0.622331	0.446559	0.712282

Cross-validation

[7]:
```
result = evaluator.compare_models_cv(models, d.X_train, d.y_train)
# result.to_csv("regression-models-cv.csv", index=False)

print("===Cross-validate result===")
display(result)
```

```
===Cross-validate result===
```

	Model	MSE	RMSE	MAE	R2
0	OLS	0.5345 ± 0.0384	0.7306 ± 0.0256	0.5315 ± 0.0068	0.5961 ± 0.0237
1	MBGD	0.5344 ± 0.0382	0.7306 ± 0.0255	0.5316 ± 0.0068	0.5961 ± 0.0235
2	WLS	0.6220 ± 0.1587	0.7829 ± 0.0953	0.5263 ± 0.0088	0.5287 ± 0.1230
3	Ridge	0.7263 ± 0.0328	0.8520 ± 0.0191	0.5972 ± 0.0149	0.4504 ± 0.0295
4	Lasso	0.7431 ± 0.0310	0.8618 ± 0.0179	0.6056 ± 0.0147	
5	Elastic Net	0.7608 ± 0.0313	0.8721 ± 0.0178	0.6158 ± 0.0151	
6	Polynomial	0.6877 ± 0.5260	0.7943 ± 0.2383	0.5201 ± 0.0120	
7	RBF	0.7295 ± 0.0219	0.8540 ± 0.0128	0.6524 ± 0.0133	
8	Fourier	1.0607 ± 2.0013	0.8303 ± 0.6093	0.4597 ± 0.0270	

```
4  0.4378 ± 0.0271
5  0.4244 ± 0.0265
6  0.4853 ± 0.3738
7  0.4479 ± 0.0249
8  0.2262 ± 1.4222
```

Test on MSE

[8]:
```
stat_result = evaluator.compare_models_statistical(
    models,
    d.X_train,
    d.y_train,
    scoring="neg_mean_squared_error"
)
# stat_result.round(4).to_csv("regression-stat-tests-mse.csv", index=False)
print("===MSE test===")
display(stat_result)
```

```
===MSE test===
```

	Model A	Model B	p-value (t-test)	p-value (Wilcoxon)
0	OLS	MBGD	8.750598e-01	0.921875
1	OLS	WLS	7.813112e-02	0.064453
2	OLS	Ridge	1.315798e-06	0.001953
3	OLS	Lasso	5.214217e-07	0.001953
4	OLS	Elastic Net	2.149158e-07	0.001953
5	OLS	Polynomial	3.746498e-01	0.625000
6	OLS	RBF	8.711911e-07	0.001953
7	OLS	Fourier	4.487288e-01	0.083984
8	MBGD	WLS	7.860186e-02	0.083984
9	MBGD	Ridge	1.260714e-06	0.001953
10	MBGD	Lasso	4.980700e-07	0.001953
11	MBGD	Elastic Net	2.048878e-07	0.001953
12	MBGD	Polynomial	3.748147e-01	0.625000
13	MBGD	RBF	8.319362e-07	0.001953
14	MBGD	Fourier	4.486860e-01	0.083984
15	WLS	Ridge	9.309441e-02	0.160156
16	WLS	Lasso	5.890877e-02	0.130859
17	WLS	Elastic Net	3.469253e-02	0.048828
18	WLS	Polynomial	6.539500e-01	0.130859
19	WLS	RBF	9.499208e-02	0.105469
20	WLS	Fourier	5.336021e-01	0.083984
21	Ridge	Lasso	1.180920e-06	0.001953
22	Ridge	Elastic Net	1.680856e-08	0.001953
23	Ridge	Polynomial	8.355328e-01	0.083984
24	Ridge	RBF	7.460160e-01	0.556641
25	Ridge	Fourier	6.248493e-01	0.083984
26	Lasso	Elastic Net	9.414249e-10	0.001953
27	Lasso	Polynomial	7.656460e-01	0.083984

28	Lasso	RBF	1.587379e-01	0.322266
29	Lasso	Fourier	6.421121e-01	0.083984
30	Elastic Net	Polynomial	6.936154e-01	0.083984
31	Elastic Net	RBF	6.768734e-03	0.003906
32	Elastic Net	Fourier	6.605441e-01	0.083984
33	Polynomial	RBF	8.199787e-01	0.083984
34	Polynomial	Fourier	6.111989e-01	0.083984
35	RBF	Fourier	6.300107e-01	0.083984

Test on MAE

[9]:
```
stat_result = evaluator.compare_models_statistical(
    models,
    d.X_train,
    d.y_train,
    scoring="neg_mean_absolute_error"
)
# stat_result.round(4).to_csv("regression-stat-tests-mae", index=False)
print("===MAE test===")
display(stat_result)
```

```
===MAE test===
```

	Model A	Model B	p-value (t-test)	p-value (Wilcoxon)
0	OLS	MBGD	2.349595e-01	0.232422
1	OLS	WLS	2.505683e-02	0.037109
2	OLS	Ridge	4.700179e-08	0.001953
3	OLS	Lasso	1.608682e-08	0.001953
4	OLS	Elastic Net	6.106149e-09	0.001953
5	OLS	Polynomial	1.314348e-02	0.037109
6	OLS	RBF	3.664756e-09	0.001953
7	OLS	Fourier	3.929934e-06	0.001953
8	MBGD	WLS	2.529498e-02	0.037109
9	MBGD	Ridge	4.590405e-08	0.001953
10	MBGD	Lasso	1.570246e-08	0.001953
11	MBGD	Elastic Net	5.974655e-09	0.001953
12	MBGD	Polynomial	1.331077e-02	0.037109
13	MBGD	RBF	3.625699e-09	0.001953
14	MBGD	Fourier	3.933100e-06	0.001953
15	WLS	Ridge	3.890623e-07	0.001953
16	WLS	Lasso	1.568528e-07	0.001953
17	WLS	Elastic Net	5.877895e-08	0.001953
18	WLS	Polynomial	3.235498e-02	0.064453
19	WLS	RBF	1.323002e-08	0.001953
20	WLS	Fourier	1.749907e-05	0.003906
21	Ridge	Lasso	4.761983e-10	0.001953
22	Ridge	Elastic Net	3.863548e-11	0.001953
23	Ridge	Polynomial	1.210329e-06	0.001953
24	Ridge	RBF	2.904943e-06	0.001953

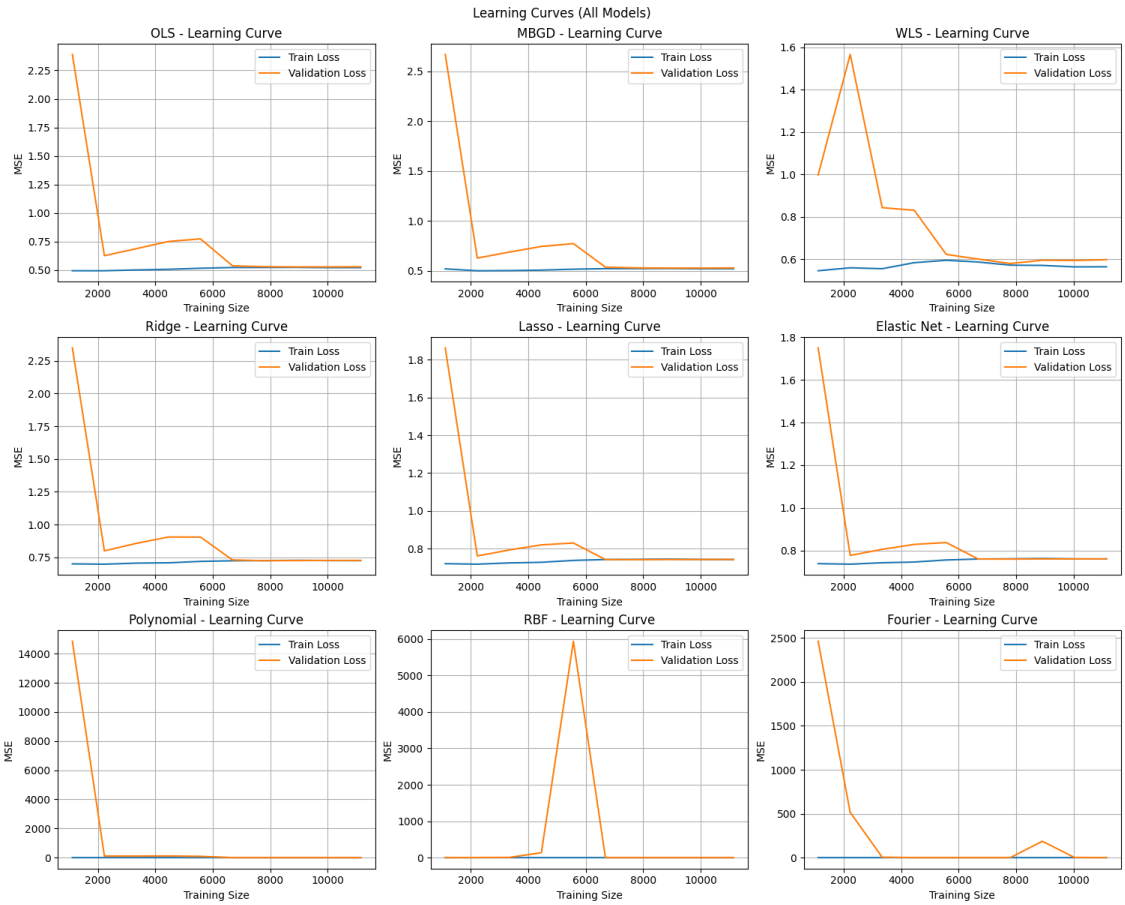
```
25          Ridge       Fourier      3.621697e-09           0.001953
26          Lasso   Elastic Net      5.329842e-10           0.001953
27          Lasso    Polynomial      4.947012e-07           0.001953
28          Lasso           RBF      9.022886e-06           0.001953
29          Lasso       Fourier      2.377962e-09           0.001953
30    Elastic Net    Polynomial      1.798505e-07           0.001953
31    Elastic Net           RBF      6.820626e-05           0.001953
32    Elastic Net       Fourier      1.180001e-09           0.001953
33     Polynomial           RBF      8.802436e-09           0.001953
34     Polynomial       Fourier      6.201350e-05           0.003906
35            RBF       Fourier      5.609359e-09           0.001953
```

1.5.2 Visualize

[10]:
```
visualizer = Visualizer()
```

Learning curves

[11]:
```
n_models = len(models)

n_cols = 3
n_rows = math.ceil(n_models / n_cols)

fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4 * n_rows),␣
 ↪layout="constrained")
axes = axes.ravel()

for i, (name, model) in enumerate(models.items()):
    ax = axes[i]

    visualizer.plot_learning_curve(
        model,
        d.X_train,
        d.y_train,
        ax=ax,
        cv=10
    )
    ax.set_title(f"{name} - Learning Curve")

# Ẩn subplot dư
for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

fig.suptitle("Learning Curves (All Models)")
plt.show()
```

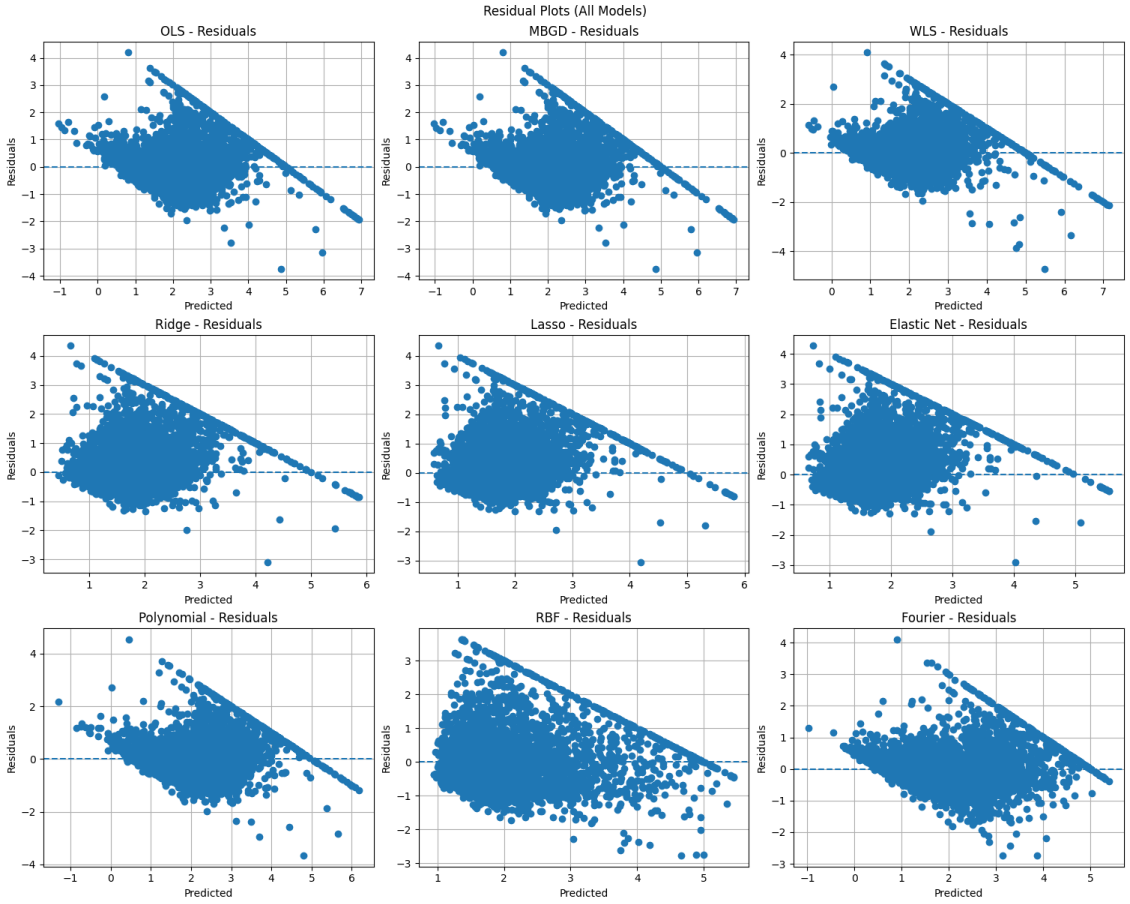


Residuals plots

[12]:
```
for name, model in models.items():
    model.fit(d.X_train, d.y_train)
```

[13]:
```
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4 * n_rows),␣
 ↪layout="constrained")
axes = axes.ravel()

for i, (name, model) in enumerate(models.items()):
    ax = axes[i]

    y_pred = model.predict(d.X_test)

    visualizer.plot_residuals(
        d.y_test,
        y_pred,
        ax=ax
    )
```

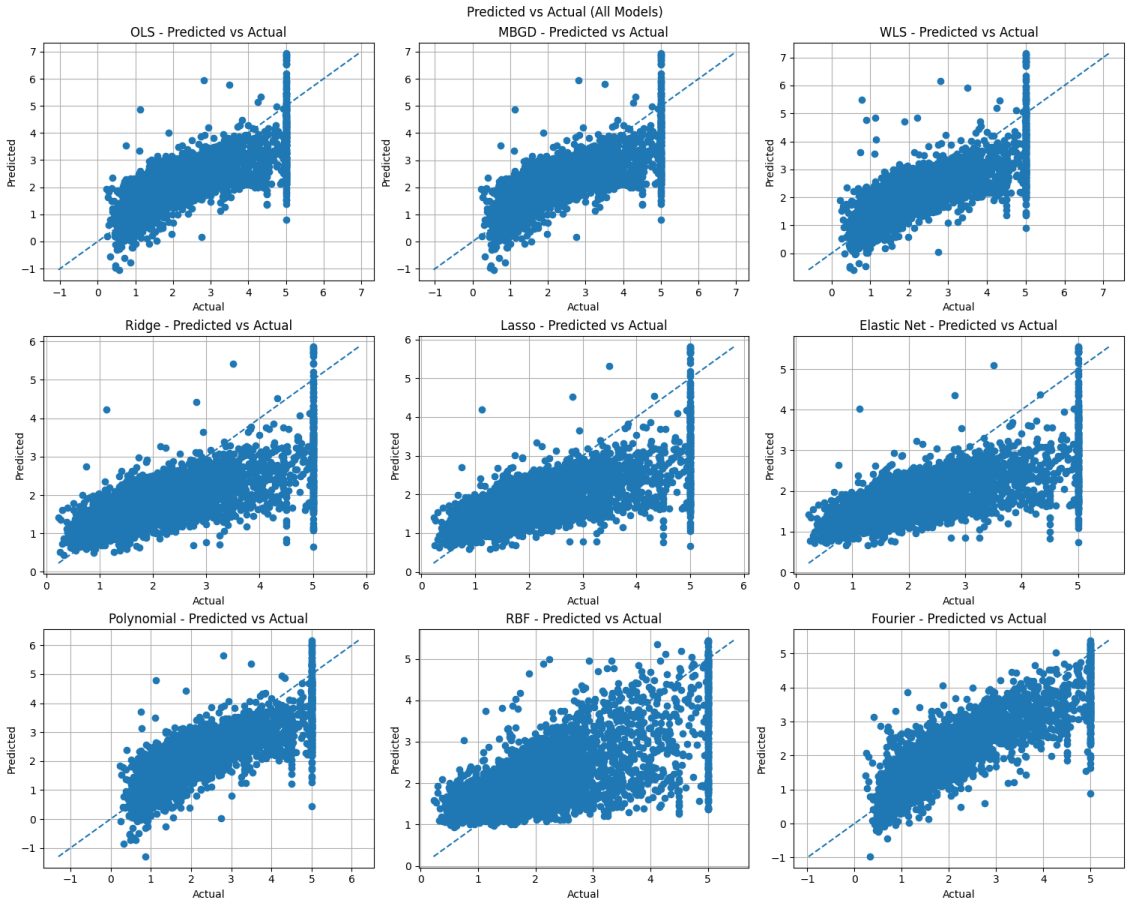
```
    ax.set_title(f"{name} - Residuals")

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

fig.suptitle("Residual Plots (All Models)")
plt.show()
```



Predicted-Actual plots

[14]:
```
fig, axes = plt.subplots(n_rows, n_cols, figsize=(15, 4 * n_rows),␣
  ↪layout="constrained")
axes = axes.ravel()

for i, (name, model) in enumerate(models.items()):
    ax = axes[i]

    y_pred = model.predict(d.X_test)
```

```
    visualizer.plot_pred_vs_actual(
        d.y_test,
        y_pred,
        ax=ax
    )
    ax.set_title(f"{name} - Predicted vs Actual")

for j in range(i + 1, len(axes)):
    axes[j].set_visible(False)

fig.suptitle("Predicted vs Actual (All Models)")
plt.show()
```



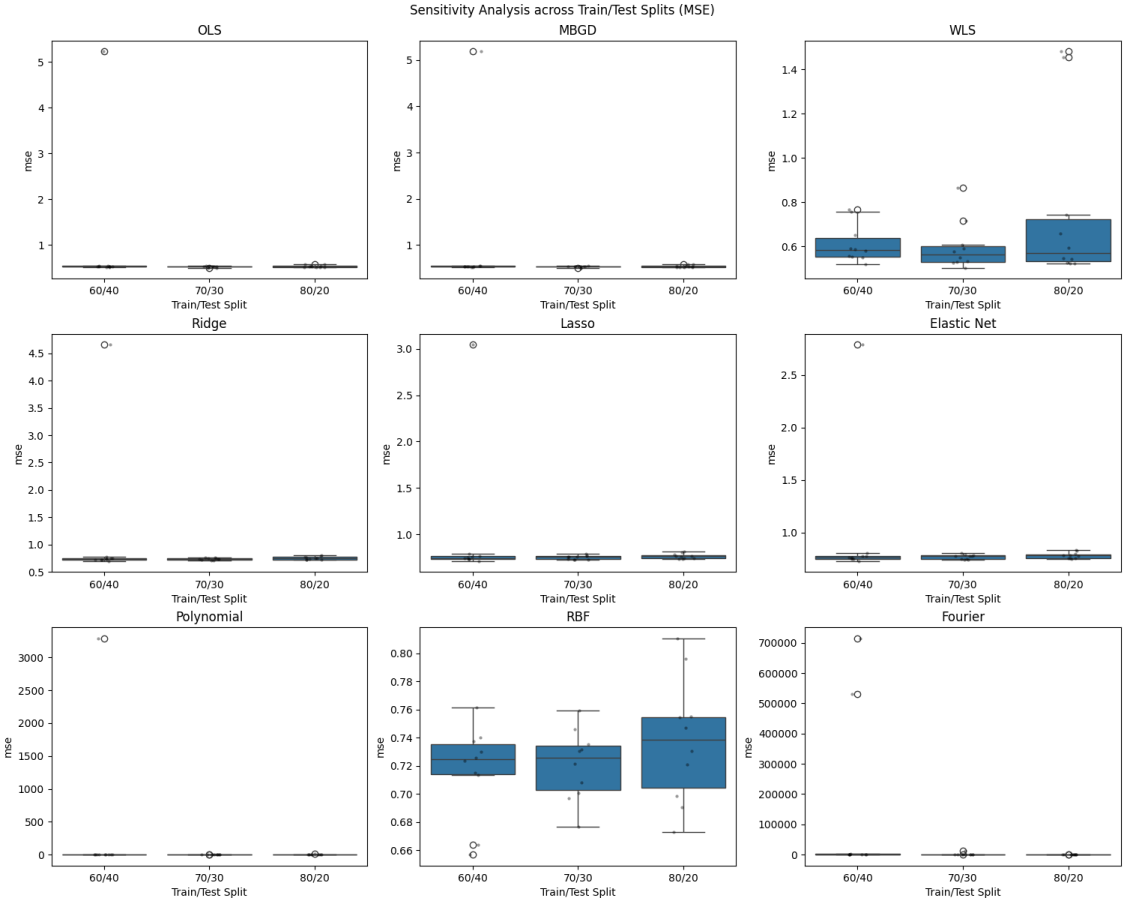
1.5.3 Sensitivity & Robustness

Split sensitivity

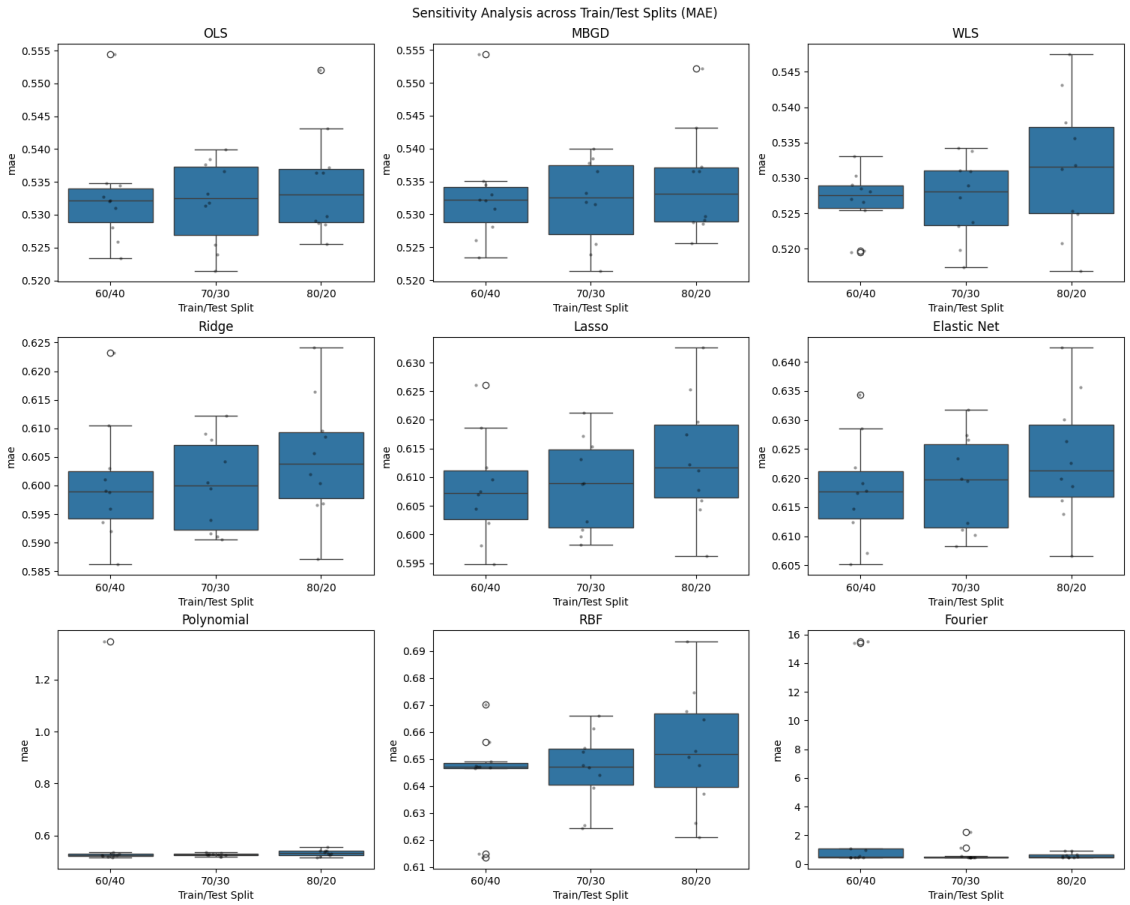
[15]:
```
df = sensitivity_split_analysis(models, copy.deepcopy(d), task="regression")
# df.round(4).to_csv("regression-sensitivity-split.csv", index=False)
display(df)
```

```
           model  split        seed         mse       rmse       mae  \
0            OLS  60/40   383329927    0.537094   0.732867  0.528066
1           MBGD  60/40   383329927    0.537145   0.732902  0.528054
2            WLS  60/40   383329927    0.581757   0.762730  0.525479
3          Ridge  60/40   383329927    0.730843   0.854894  0.599106
4          Lasso  60/40   383329927    0.747263   0.864444  0.606995
..           …    …         …          …         …         …
265        Lasso  80/20  1904449481    0.809111   0.899506  0.625324
266  Elastic Net  80/20  1904449481    0.827224   0.909518  0.635595
267   Polynomial  80/20  1904449481    0.841337   0.917244  0.539934
268          RBF  80/20  1904449481    0.795880   0.892121  0.674723
269      Fourier  80/20  1904449481  454.001992  21.307322  0.926883

              r2
0       0.597936
1       0.597898
2       0.564502
3       0.452897
4       0.440606
..           …
265     0.409906
266     0.396696
267     0.386403
268     0.419556
269  -330.108886

[270 rows x 7 columns]
```

[16]:
```
fig = plot_split_boxplots(df, metric="mse")
plt.show()
```



[30]:
```
fig = plot_split_boxplots(df, metric="mae")
plt.show()
```



Gaussian noise sensitivity

[18]:
```
result = noise_injection_analysis(models, copy.deepcopy(d), task="regression")
# result.round(4).to_csv("regression-sensitivity-gaussian-noise.csv",␣
  ↪index=False)
display(result)
```

	model	sigma	mse	rmse	mae	r2	is_clean	\
0	OLS	0.0	0.524912	0.724508	0.529685	0.610048	True	
1	MBGD	0.0	0.525089	0.724630	0.529841	0.609917	True	
2	WLS	0.0	0.531956	0.729353	0.522687	0.604816	True	
3	Ridge	0.0	0.730986	0.854977	0.596158	0.456958	True	
4	Lasso	0.0	0.748882	0.865380	0.605771	0.443663	True	
5	Elastic Net	0.0	0.767259	0.875933	0.615911	0.430011	True	
6	Polynomial	0.0	0.504271	0.710120	0.519910	0.625382	True	
7	RBF	0.0	0.657261	0.810716	0.602315	0.511728	True	
8	Fourier	0.0	0.387824	0.622755	0.446926	0.711890	True	
9	OLS	0.1	0.536959	0.732775	0.536539	0.601099	False	
10	MBGD	0.1	0.537044	0.732833	0.536614	0.601036	False	
11	WLS	0.1	0.539911	0.734786	0.532073	0.598906	False	

12	Ridge	0.1	0.734153	0.856827	0.597953	0.454606	False
13	Lasso	0.1	0.751972	0.867163	0.607581	0.441367	False
14	Elastic Net	0.1	0.770322	0.877680	0.617706	0.427736	False
15	Polynomial	0.1	0.521826	0.722375	0.530658	0.612341	False
16	RBF	0.1	0.722539	0.850023	0.646775	0.463233	False
17	Fourier	0.1	0.403913	0.635541	0.455313	0.699937	False
18	OLS	0.5	0.629569	0.793454	0.591160	0.532300	False
19	MBGD	0.5	0.629589	0.793466	0.591180	0.532285	False
20	WLS	0.5	0.631569	0.794713	0.592103	0.530814	False
21	Ridge	0.5	0.775643	0.880706	0.624053	0.423783	False
22	Lasso	0.5	0.795731	0.892037	0.633573	0.408860	False
23	Elastic Net	0.5	0.812797	0.901553	0.641988	0.396181	False
24	Polynomial	0.5	0.622747	0.789143	0.590889	0.537368	False
25	RBF	0.5	0.797052	0.892778	0.687134	0.407878	False
26	Fourier	0.5	0.559267	0.747842	0.548892	0.584526	False
27	OLS	1.0	0.779405	0.882839	0.676395	0.420988	False
28	MBGD	1.0	0.779450	0.882864	0.676417	0.420955	False
29	WLS	1.0	0.787604	0.887470	0.679795	0.414897	False
30	Ridge	1.0	0.884239	0.940340	0.683933	0.343108	False
31	Lasso	1.0	0.904405	0.951002	0.692338	0.328127	False
32	Elastic Net	1.0	0.918640	0.958457	0.698539	0.317552	False
33	Polynomial	1.0	0.769613	0.877276	0.668498	0.428263	False
34	RBF	1.0	0.847871	0.920799	0.713502	0.370125	False
35	Fourier	1.0	0.766375	0.875429	0.662221	0.430668	False

	mse_sens_abs	mse_sens_rel	rmse_sens_abs	rmse_sens_rel	mae_sens_abs	\
0	NaN	NaN	NaN	NaN	NaN	
1	NaN	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	NaN	
5	NaN	NaN	NaN	NaN	NaN	
6	NaN	NaN	NaN	NaN	NaN	
7	NaN	NaN	NaN	NaN	NaN	
8	NaN	NaN	NaN	NaN	NaN	
9	0.012047	0.022950	0.008267	0.011410	0.006854	
10	0.011954	0.022767	0.008202	0.011319	0.006773	
11	0.007955	0.014954	0.005433	0.007449	0.009387	
12	0.003166	0.004332	0.001850	0.002163	0.001795	
13	0.003091	0.004127	0.001784	0.002061	0.001809	
14	0.003063	0.003992	0.001746	0.001994	0.001795	
15	0.017555	0.034812	0.012255	0.017257	0.010748	
16	0.065278	0.099318	0.039307	0.048484	0.044461	
17	0.016089	0.041486	0.012786	0.020532	0.008387	
18	0.104656	0.199378	0.068945	0.095161	0.061475	
19	0.104499	0.199013	0.068836	0.094994	0.061339	
20	0.099613	0.187258	0.065360	0.089614	0.069416	
21	0.044656	0.061091	0.025728	0.030093	0.027895	

22	0.046849	0.062558	0.026658	0.030805	0.027802
23	0.045538	0.059351	0.025619	0.029248	0.026077
24	0.118476	0.234945	0.079023	0.111281	0.070979
25	0.139791	0.212687	0.082061	0.101221	0.084819
26	0.171444	0.442066	0.125087	0.200860	0.101966
27	0.254492	0.484828	0.158331	0.218535	0.146711
28	0.254360	0.484414	0.158234	0.218365	0.146576
29	0.255648	0.480581	0.158117	0.216791	0.157108
30	0.153253	0.209652	0.085363	0.099842	0.087775
31	0.155523	0.207674	0.085623	0.098942	0.086566
32	0.151381	0.197301	0.082524	0.094212	0.082628
33	0.265342	0.526189	0.167155	0.235390	0.148589
34	0.190610	0.290007	0.110083	0.135785	0.111187
35	0.378552	0.976092	0.252674	0.405735	0.215295

	mae_sens_rel	r2_sens_abs	r2_sens_rel
0	NaN	NaN	NaN
1	NaN	NaN	NaN
2	NaN	NaN	NaN
3	NaN	NaN	NaN
4	NaN	NaN	NaN
5	NaN	NaN	NaN
6	NaN	NaN	NaN
7	NaN	NaN	NaN
8	NaN	NaN	NaN
9	0.012941	0.008949	0.014670
10	0.012784	0.008881	0.014561
11	0.017958	0.005909	0.009771
12	0.003011	0.002352	0.005148
13	0.002987	0.002296	0.005175
14	0.002915	0.002275	0.005291
15	0.020672	0.013041	0.020853
16	0.073816	0.048494	0.094766
17	0.018765	0.011952	0.016790
18	0.116060	0.077748	0.127446
19	0.115769	0.077631	0.127282
20	0.132806	0.074001	0.122354
21	0.046791	0.033175	0.072599
22	0.045895	0.034803	0.078446
23	0.042339	0.033830	0.078672
24	0.136521	0.088015	0.140737
25	0.140822	0.103849	0.202939
26	0.228149	0.127364	0.178909
27	0.276977	0.189060	0.309910
28	0.276641	0.188962	0.309815
29	0.300578	0.189918	0.314010
30	0.147235	0.113850	0.249148
31	0.142903	0.115537	0.260415

```
32      0.134155     0.112459     0.261526
33      0.285797     0.197120     0.315199
34      0.184600     0.141602     0.276715
35      0.481724     0.281222     0.395036
```

Feature corruption with imputers sensitivity

[19]:
```
result = feature_corruption_analysis(models, copy.deepcopy(d),␣
  ↪task="regression")
# result.round(4).to_csv("regression-sensitivity-imputer.csv", index=False)
display(result)
```

```
          model imputer  missing_rate  is_clean       mse      rmse       mae  \
0           OLS    none           0.0      True  0.524912  0.724508  0.529685
1          MBGD    none           0.0      True  0.525089  0.724630  0.529841
2           WLS    none           0.0      True  0.531956  0.729353  0.522687
3         Ridge    none           0.0      True  0.730986  0.854977  0.596158
4         Lasso    none           0.0      True  0.748882  0.865380  0.605771
..          …     …            …        …       …       …       …
85        Lasso     knn           0.3     False  0.964835  0.982260  0.698308
86  Elastic Net     knn           0.3     False  0.976865  0.988365  0.704649
87   Polynomial     knn           0.3     False  0.851371  0.922698  0.687700
88          RBF     knn           0.3     False  0.937366  0.968176  0.737136
89      Fourier     knn           0.3     False  0.694425  0.833322  0.618247

          r2  mse_sens_abs  mse_sens_rel  rmse_sens_abs  rmse_sens_rel  \
0   0.610048           NaN           NaN            NaN            NaN
1   0.609917           NaN           NaN            NaN            NaN
2   0.604816           NaN           NaN            NaN            NaN
3   0.456958           NaN           NaN            NaN            NaN
4   0.443663           NaN           NaN            NaN            NaN
..       …           …           …              …              …
85  0.283234      0.215954      0.288368       0.116881       0.135063
86  0.274297      0.209605      0.273187       0.112431       0.128356
87  0.367525      0.347100      0.688320       0.212577       0.299354
88  0.303641      0.280105      0.426169       0.157460       0.194223
89  0.484119      0.306601      0.790569       0.210567       0.338121

    mae_sens_abs  mae_sens_rel  r2_sens_abs  r2_sens_rel
0            NaN           NaN          NaN          NaN
1            NaN           NaN          NaN          NaN
2            NaN           NaN          NaN          NaN
3            NaN           NaN          NaN          NaN
4            NaN           NaN          NaN          NaN
..           …           …          …          …
85      0.092537      0.152759     0.160430     0.361602
86      0.088738      0.144077     0.155714     0.362115
87      0.167790      0.322730     0.257857     0.412319
88      0.134821      0.223838     0.208087     0.406636
```

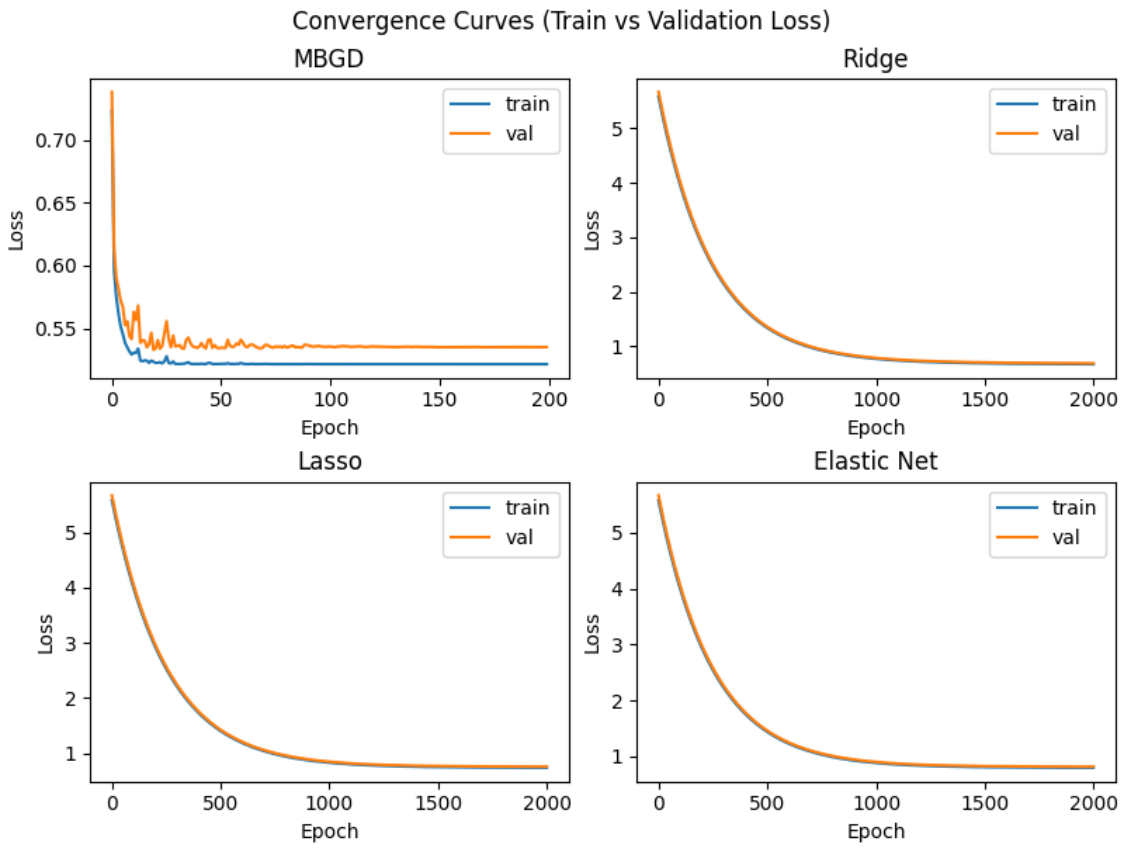
```
89      0.171321      0.383331      0.227771      0.319953

[90 rows x 16 columns]
```

Loss curves

[20]:
```
gd_models = {
    name: model
    for name, model in models.items()
    if isinstance(model.named_steps["predictor"], BaseGDModel)
}
display(gd_models)
```

```
{'MBGD': Pipeline(steps=[('scaler', StandardScaler()),
                 ('predictor', MBGD(lr_sched='step_decay'))]),
 'Ridge': Pipeline(steps=[('scaler', StandardScaler()),
                 ('predictor', RidgeRegression(alpha=0.1))]),
 'Lasso': Pipeline(steps=[('scaler', StandardScaler()),
                 ('predictor', LassoRegression(alpha=0.1))]),
 'Elastic Net': Pipeline(steps=[('scaler', StandardScaler()),
                 ('predictor',
                  ElasticNet(alpha_1=np.float64(0.1), alpha_2=np.float64(0.
 ↪1)))])}
```

[31]:
```
epochs = [200, 2000, 2000, 2000]
fig = plot_loss_curves(gd_models, copy.deepcopy(d), epochs)
fig.show()
```



1.5.4 Benchmarks

[22]:
```
bm_models = {
    name: model
    for name, model in models.items()
    if isinstance(model.named_steps["predictor"], BaseModel)
}
display(bm_models)
```

```
{'OLS': Pipeline(steps=[('scaler', StandardScaler()), ('predictor', OLS())]),
 'MBGD': Pipeline(steps=[('scaler', StandardScaler()),
                 ('predictor', MBGD(lr_sched='step_decay'))]),
 'WLS': Pipeline(steps=[('scaler', StandardScaler()), ('predictor', WLS())]),
 'Ridge': Pipeline(steps=[('scaler', StandardScaler()),
                 ('predictor', RidgeRegression(alpha=0.1))]),
 'Lasso': Pipeline(steps=[('scaler', StandardScaler()),
                 ('predictor', LassoRegression(alpha=0.1))]),
 'Elastic Net': Pipeline(steps=[('scaler', StandardScaler()),
                 ('predictor',
                  ElasticNet(alpha_1=np.float64(0.1), alpha_2=np.float64(0.
↪1)))]),
```

```
'Polynomial': Pipeline(steps=[('scaler', StandardScaler()),
                ('features', PolynomialBasis(degree=2)), ('predictor', OLS())]),
'RBF': Pipeline(steps=[('scaler', StandardScaler()), ('features', RBF()),
                ('predictor', OLS())]),
'Fourier': Pipeline(steps=[('scaler', StandardScaler()), ('features',␣
↪FourierBasis()),
                ('predictor', OLS())])}
```

[23]:
```
result = benchmark_models(bm_models, d)
# result.round(4).to_csv("regression-benchmark-models.csv")
display(result)
```

	model	train_time_mean	train_time_std	train_memory_mean	train_memory_std	inference_time_mean	inference_time_std	inference_memory_mean	inference_memory_std
0	OLS	0.003085	0.001708	1329249.8	3686.653898	0.000251	3.497052e-04	68195.4	2725.415682
1	MBGD	2.063587	0.023515	2518517.6	1887.457719	0.000026	3.316696e-06	66368.0	0.000000
2	WLS	0.004419	0.000229	4758065.0	0.000000	0.000030	2.978570e-06	66344.0	0.000000
3	Ridge	0.592653	0.007809	2511564.8	249.600000	0.000021	9.086107e-07	66344.0	0.000000
4	Lasso	0.589782	0.004823	2511440.0	0.000000	0.000022	7.844962e-07	66344.0	0.000000
5	Elastic Net	0.606384	0.008371	2511472.0	0.000000	0.000023	3.794837e-06	66344.0	0.000000
6	Polynomial	0.001199	0.000009	2378280.0	0.000000	0.000040	2.681506e-06	66344.0	0.000000
7	RBF	0.001081	0.000142	1585704.0	0.000000	0.000059	8.575019e-06	66344.0	0.000000
8	Fourier	0.006999	0.000034	10966401.0	0.000000	0.000213	1.290557e-05	66344.0	0.000000

[ ]:
```

```